

olist_negative_sentiment_geometry_analysis

May 26, 2026

1 Geometric Analysis of Negative Sentiment in Ecommerce Reviews Across Embedding Models

Tram Anh Hoang - MA in Data Science for Economics

1.1 1 - Data Preparation

This section prepares the review corpus for geometric analysis by cleaning text, standardizing format, and building baseline sentiment labels.

1.1.1 1.1 Set Up

This part initializes configuration, paths, and shared modules so the pipeline runs reproducibly end to end.

```
[8]: from __future__ import annotations

from pathlib import Path
import sys
import warnings

import numpy as np
import pandas as pd
import plotly.express as px

import json
import logging
import importlib
import importlib.util
import inspect

# Resolve package root before importing local src modules.
PROJECT_ROOT = Path.cwd().resolve()
if (PROJECT_ROOT / "submission" / "src").exists():
    PROJECT_ROOT = PROJECT_ROOT / "submission"
elif not (PROJECT_ROOT / "src").exists() and (PROJECT_ROOT.parent / "src").
    exists():
    PROJECT_ROOT = PROJECT_ROOT.parent
```

```

if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))

import src.projection as projection_module

from src.notebook_text_helpers import (
    load_portuguese_stopwords,
    top_terms,
    text_stats,
    score_review,
    sample_examples,
)
from src.notebook_neighbors import (
    inspect_neighbors,
    summarize_neighbors,
    plot_local_neighbourhood,
)
from src.notebook_phase3_helpers import (
    SemanticEmbedder,
    GeometryClusteringEvaluator,
    load_cached_l2,
    read_csv_or_empty,
    run_k_grid_diagnostics,
    stratified_sentiment_indices,
    concat_by_field,
    run_phase3_for_model,
)
from src.notebook_projection_helpers import (
    collect_method_frame,
    ensure_model_labels,
    plot_method_panels,
)
from src.notebook_rq_helpers import (
    bootstrap_cluster_neg_std,
    dispersion_profile,
    negative_homophily,
)

warnings.filterwarnings("ignore")
pd.set_option("display.max_colwidth", 140)

```

```

[9]: if str(PROJECT_ROOT) not in sys.path:
        sys.path.append(str(PROJECT_ROOT))

DATA_DIR = PROJECT_ROOT / "data"
ARTIFACTS_DIR = PROJECT_ROOT / "artifacts"

```

```

PROCESSED_DIR = DATA_DIR / "processed"
PROCESSED_DIR.mkdir(parents=True, exist_ok=True)
PREPROCESSED_PKL = PROCESSED_DIR / "reviews_preprocessed.pkl"

# Minimal path fallbacks (only when submission/data is partial)
LEXICON_DIR = DATA_DIR / "lexicons"
if not LEXICON_DIR.exists():
    LEXICON_DIR = PROJECT_ROOT.parent / "data" / "lexicons"

EMBED_DIR = DATA_DIR / "embeddings"
if not EMBED_DIR.exists():
    EMBED_DIR = PROJECT_ROOT.parent / "data" / "embeddings"
EMBED_DIR.mkdir(parents=True, exist_ok=True)

from src.data_loader import OlistReviewLoader
from src import preprocessor as preprocessor_module
from src.preprocessor import PortugueseTextPreprocessor
from src.projection import ProjectionEngine
from src.topic_modeler import ClusterTopicModeler
from src.evaluator import GeometryEvaluator
from src.quality import DataQualityAuditor
from src.reproducibility import ArtifactStore, set_global_seed
from src.experiment_config import ExperimentConfig

print(f"Project root: {PROJECT_ROOT}")
print(f>Data directory: {DATA_DIR}")
print(f"Artifacts directory: {ARTIFACTS_DIR}")
print(f"Preprocessed cache path: {PREPROCESSED_PKL}")
print(f"Lexicon directory: {LEXICON_DIR}")
print(f"Embedding cache directory: {EMBED_DIR}")
print(f"Preprocessor module: {preprocessor_module.__file__}")

```

```

Project root: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-
Semantic-P15\submission
Data directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-
Semantic-P15\submission\data
Artifacts directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\submission\artifacts
Preprocessed cache path: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\submission\data\processed\reviews_preprocessed.pkl
Lexicon directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\data\lexicons
Embedding cache directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\data\embeddings
Preprocessor module: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\submission\src\preprocessor.py

```

```
[10]: # Shared text-helper imports are centralized in the setup cell.
```

```
[11]: cfg = ExperimentConfig()
      cfg.validate()

      set_global_seed(cfg.random_state)
      store = ArtifactStore(root_dir=ARTIFACTS_DIR, run_name="analysis-v1")
      run_dir = store.ensure()

      print(f"Run directory: {run_dir}")
      print(f"Available models: {cfg.model_keys} | selected: {cfg.model_key}")
      print(f"Available projections: {cfg.projection_methods} | selected: {cfg.
        ↪projection_method}")
      cfg
```

```
Run directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-
Semantic-P15\submission\artifacts\analysis-v1\20260513-133515
Available models: ('bertimbau', 'multilingual_minilm', 'openclip_text') |
selected: multilingual_minilm
Available projections: ('pca', 'umap', 'tsne') | selected: umap
```

```
[11]: ExperimentConfig(audit_sample_size=None, sample_size=None, min_chars=None,
min_retention_target=0.9, random_state=42, stability_seeds=(11, 21, 42, 84),
model_keys=('bertimbau', 'multilingual_minilm', 'openclip_text'),
model_key='multilingual_minilm', projection_methods=('pca', 'umap', 'tsne'),
projection_method='umap', k_grid=(2, 4, 6, 8, 10, 12))
```

```
[12]: loader = OlistReviewLoader(DATA_DIR)
      auditor = DataQualityAuditor()

      raw_reviews = loader.load_reviews(
          require_text=False,
          sample_size=cfg.audit_sample_size,
          random_state=cfg.random_state,
      )

      overview_df = auditor.dataset_overview(raw_reviews, id_col="review_id")
      text_quality_df = auditor.text_quality_report(
          raw_reviews,
          text_col="review_text",
          score_col="review_score",
      )

      display(overview_df)
      display(text_quality_df)
```

	metric	value
0	n_rows	98410.0

1	n_columns	8.0
2	duplicate_rows	0.0
3	duplicate_ratio	0.0
4	distinct_review_id	98410.0

	metric	value
0	rows_total	98410.000000
1	rows_non_empty_text	40641.000000
2	text_non_empty_ratio	0.412976
3	text_duplicate_ratio_non_empty	0.123644
4	text_length_mean	28.233000
5	text_length_median	0.000000
6	token_length_mean	4.824000
7	distinct_ids	98410.000000
8	review_score_missing_ratio	0.000000

1.1.2 1.2 Text Normalization

This part reduces textual noise and applies consistent normalization before feature extraction.

1.2.1 Text Parsing and Regex We remove empty or low-information entries and standardize text structure for downstream token processing.

```
[13]: # preliminary cleaning: normalize text + remove non-informative rows
rows_before = len(raw_reviews)

raw_reviews["review_text"] = (
    raw_reviews["review_text"]
    .fillna("")
    .astype(str)
    .str.lower()
    .str.replace(r"https?:\/\/\S+|www\.\S+", " ", regex=True)
    .str.replace(r"<[^>]+>", " ", regex=True)
    .str.replace(r"\d+", " ", regex=True)
    .str.replace(r"^[a-zâ-öø-ÿ\s]", " ", regex=True)
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)

# Drop empty text after cleaning and deduplicate reviews
raw_reviews = raw_reviews[raw_reviews["review_text"].str.len() > 0].copy()
if "review_id" in raw_reviews.columns:
    raw_reviews = raw_reviews.drop_duplicates(subset=["review_id"])
raw_reviews = raw_reviews.reset_index(drop=True)

rows_after = len(raw_reviews)
cleaning_report = pd.DataFrame(
    [
```

```

        {"metric": "rows_before_cleaning", "value": int(rows_before)},
        {"metric": "rows_after_cleaning", "value": int(rows_after)},
        {"metric": "rows_removed", "value": int(rows_before - rows_after)},
    ]
)
display(cleaning_report)

```

	metric	value
0	rows_before_cleaning	98410
1	rows_after_cleaning	40482
2	rows_removed	57928

```

[14]: # Recompute quality tables after 1.2 cleaning
missing_df = auditor.missingness_report(raw_reviews)
score_dist_df = auditor.score_distribution(raw_reviews)

display(missing_df.head(12))
display(score_dist_df)

fig_scores = px.bar(
    score_dist_df,
    x="review_score",
    y="count",
    color="review_score",
    title="Review score distribution after 1.2 cleaning",
    height=380,
)
fig_scores.show()

```

	column	missing_ratio
0	review_comment_title	0.75935
1	review_id	0.00000
2	order_id	0.00000
3	review_score	0.00000
4	review_comment_message	0.00000
5	review_creation_date	0.00000
6	review_answer_timestamp	0.00000
7	review_text	0.00000

	review_score	count	ratio
0	1	8632	0.213231
1	2	2115	0.052245
2	3	3518	0.086903
3	4	5891	0.145521
4	5	20326	0.502100

Detailed interpretation moved to report; notebook keeps only execution outputs.

1.2.2 Tokenization We inspect token-length and lexical patterns to understand how much linguistic signal is retained.

```
[15]: eda_text_df = raw_reviews[["review_score", "review_text"]].copy()
eda_text_df["review_text"] = eda_text_df["review_text"].fillna("").astype(str).
    ↪str.strip()
eda_text_df = eda_text_df[eda_text_df["review_text"] != ""].
    ↪reset_index(drop=True)

eda_text_df["char_len"] = eda_text_df["review_text"].str.len()
eda_text_df["token_len"] = eda_text_df["review_text"].str.split().map(len)

eda_text_df["score_band"] = np.select(
    [
        eda_text_df["review_score"] <= 2,
        eda_text_df["review_score"] == 3,
        eda_text_df["review_score"] >= 4,
    ],
    ["low_1_2", "mid_3", "high_4_5"],
    default="unknown",
)

eda_length_summary = (
    eda_text_df.groupby("score_band")
    .agg(
        n_reviews=("review_text", "size"),
        mean_char_len=("char_len", "mean"),
        median_char_len=("char_len", "median"),
        mean_token_len=("token_len", "mean"),
        median_token_len=("token_len", "median"),
    )
    .reset_index()
    .sort_values("n_reviews", ascending=False)
)

display(eda_length_summary.round(2))

fig_len_box = px.box(
    eda_text_df.sample(min(30000, len(eda_text_df)), random_state=cfg.
    ↪random_state),
    x="score_band",
    y="token_len",
    color="score_band",
    points=False,
    title="Token-length distribution by review score band",
    height=420,
)
```

```
fig_len_box.show()
```

	score_band	n_reviews	mean_char_len	median_char_len	mean_token_len	\
0	high_4_5	26217	52.13	41.0	8.93	
1	low_1_2	10747	95.44	85.0	17.16	
2	mid_3	3518	80.35	67.0	14.53	

	median_token_len
0	7.0
1	15.0
2	12.0

1.2.3 Stop-word Removal We filter frequent Portuguese function words to emphasize semantically informative terms.

```
[16]: pt_stopwords = load_portuguese_stopwords()

top_unigrams_all = top_terms(eda_text_df["review_text"], ngram_range=(1, 1),
    ↪min_df=25, top_n=20, pt_stopwords=pt_stopwords)
top_bigrams_all = top_terms(eda_text_df["review_text"], ngram_range=(2, 2),
    ↪min_df=20, top_n=20, pt_stopwords=pt_stopwords)
top_unigrams_low = top_terms(eda_text_df.loc[eda_text_df["score_band"] ==
    ↪"low_1_2", "review_text"], ngram_range=(1, 1), min_df=10, top_n=20,
    ↪pt_stopwords=pt_stopwords)
top_unigrams_high = top_terms(eda_text_df.loc[eda_text_df["score_band"] ==
    ↪"high_4_5", "review_text"], ngram_range=(1, 1), min_df=10, top_n=20,
    ↪pt_stopwords=pt_stopwords)

term_contrast = (
    top_unigrams_low.rename(columns={"count": "count_low"})
    .merge(top_unigrams_high.rename(columns={"count": "count_high"}),
    ↪on="term", how="outer")
    .fillna(0)
)
term_contrast["delta_low_minus_high"] = term_contrast["count_low"] -
    ↪term_contrast["count_high"]
term_contrast = term_contrast.sort_values("delta_low_minus_high",
    ↪ascending=False).reset_index(drop=True)

display(top_unigrams_all)
display(top_bigrams_all)
display(term_contrast.head(20))
```

	term	count
0	produto	18290
1	não	10693
2	prazo	8431
3	muito	7883

4	entrega	6484
5	antes	5605
6	chegou	5523
7	recebi	5211
8	bom	4573
9	foi	4518
10	recomendo	4314
11	entregue	3753
12	veio	3250
13	bem	3103
14	mas	2965
15	qualidade	2757
16	comprei	2721
17	loja	2397
18	eu	2397
19	tudo	2315

	term	count
0	antes prazo	4157
1	muito bom	2459
2	não recebi	1800
3	foi entregue	1687
4	recebi produto	1599
5	produto chegou	1513
6	chegou antes	1335
7	ainda não	1201
8	dentro prazo	1075
9	produto não	984
10	bem antes	969
11	produto entregue	875
12	não foi	802
13	meu produto	799
14	produto veio	677
15	entregue antes	677
16	prazo entrega	671
17	entrega rápida	659
18	produto muito	658
19	bem embalado	652

	term	count_low	count_high	delta_low_minus_high
0	não	7412.0	1930.0	5482.0
1	comprei	1848.0	0.0	1848.0
2	recebi	3226.0	1450.0	1776.0
3	veio	1525.0	0.0	1525.0
4	ainda	1384.0	0.0	1384.0
5	meu	1307.0	0.0	1307.0
6	até	1178.0	0.0	1178.0
7	mas	1011.0	0.0	1011.0
8	estou	948.0	0.0	948.0

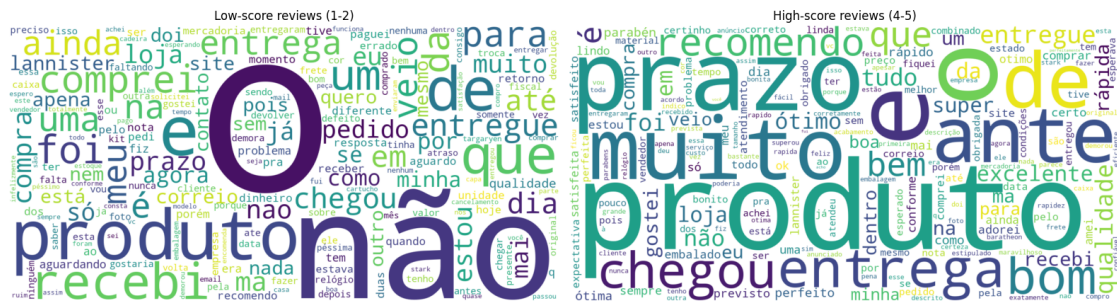
9	me	935.0	0.0	935.0
10	nao	935.0	0.0	935.0
11	compra	906.0	0.0	906.0
12	eu	902.0	0.0	902.0
13	foi	1945.0	2091.0	-146.0
14	entregue	1314.0	2157.0	-843.0
15	gostei	0.0	1459.0	-1459.0
16	loja	0.0	1464.0	-1464.0
17	super	0.0	1619.0	-1619.0
18	ótimo	0.0	1748.0	-1748.0
19	excelente	0.0	1757.0	-1757.0

```
[17]: from wordcloud import WordCloud
import matplotlib.pyplot as plt

low_blob = " ".join(eda_text_df.loc[eda_text_df["score_band"] == "low_1_2",
    ↪ "review_text"].astype(str).tolist())
high_blob = " ".join(eda_text_df.loc[eda_text_df["score_band"] == "high_4_5",
    ↪ "review_text"].astype(str).tolist())

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
for ax, blob, title in [
    (axes[0], low_blob, "Low-score reviews (1-2)",
    (axes[1], high_blob, "High-score reviews (4-5)",
]:
    if blob.strip():
        wc = WordCloud(width=1400, height=700, background_color="white",
    ↪ collocations=False).generate(blob)
        ax.imshow(wc, interpolation="bilinear")
        ax.set_title(title)
        ax.axis("off")

plt.tight_layout()
plt.show()
```



1.2.4 Text-length threshold We compare candidate minimum-length rules and retain a threshold that balances quality and sample size.

```
[18]: threshold_rows = []
text_all = raw_reviews["review_text"].fillna("").astype(str)
non_empty_base = int((text_all.str.len() >= 4).sum())

for threshold in [0, 4, 8, 12, 16, 24, 32]:
    keep = text_all.str.len() >= threshold
    rows_kept = int(keep.sum())

    threshold_rows.append(
        {
            "min_chars": threshold,
            "rows_kept": rows_kept,
            "retention_ratio_overall": float(rows_kept / len(raw_reviews)) if
↪len(raw_reviews) else np.nan,
            "retention_vs_non_empty_base": float(rows_kept / non_empty_base) if
↪non_empty_base else np.nan,
            "avg_text_len": float(text_all[keep].str.len().mean()) if rows_kept
↪else np.nan,
        }
    )

threshold_df = pd.DataFrame(threshold_rows)
display(threshold_df)
```

	min_chars	rows_kept	retention_ratio_overall	retention_vs_non_empty_base \
0	0	40482	1.000000	1.017929
1	4	39769	0.982387	1.000000
2	8	38623	0.954078	0.971184
3	12	36567	0.903290	0.919485
4	16	35180	0.869028	0.884609
5	24	31915	0.788375	0.802509
6	32	28339	0.700040	0.712590

	avg_text_len
0	66.078998
1	67.215293
2	69.049090
3	72.408921
4	74.728027
5	80.378192
6	87.039698

```
[19]: fig_threshold = px.line(
    threshold_df,
    x="min_chars",
```

```

    y=["retention_ratio_overall", "retention_vs_non_empty_base"],
    markers=True,
    title="Retention by text-length threshold (overall and vs non-empty_
↳baseline)",
    height=420,
)
fig_threshold.show()

# Data-driven threshold choice (unless manual override is provided in cfg.
↳min_chars)
if cfg.min_chars is not None:
    selected_min_chars = int(cfg.min_chars)
    selection_reason = "manual override from cfg.min_chars"
else:
    # Exclude min_chars < 4 from automatic selection to avoid keeping empty/
↳micro text.
    candidates = threshold_df[threshold_df["min_chars"] >= 4]
    eligible = candidates[candidates["retention_vs_non_empty_base"] >= cfg.
↳min_retention_target]

    if eligible.empty:
        selected_min_chars = int(candidates.
↳loc[candidates["retention_vs_non_empty_base"].idxmax(), "min_chars"])
        selection_reason = (
            f"fallback: no threshold reached target retention {cfg.
↳min_retention_target:.0%} "
            "relative to non-empty baseline"
        )
    else:
        selected_min_chars = int(eligible["min_chars"].max())
        selection_reason = (
            f"largest threshold that keeps at least {cfg.min_retention_target:.
↳0%} "
            "of non-empty baseline reviews"
        )

print(f"Selected min_chars = {selected_min_chars} ({selection_reason})")

```

Selected min_chars = 12 (largest threshold that keeps at least 90% of non-empty baseline reviews)

1.2.5 Stemming / Lemmatization Diagnostics We compare stemmed and non-stemmed outputs to verify whether stemming improves compactness without harming interpretability.

Detailed interpretation moved to report; notebook keeps only execution outputs.

```
[20]: # Use full dataset
df = raw_reviews[["review_text"]].copy()
df["review_text"] = df["review_text"].fillna("").astype(str)

# Preprocessors
prep_no_stem = PortugueseTextPreprocessor(remove_stopwords=True,
    ↪use_stemming=False, min_token_length=2)
prep_stem = PortugueseTextPreprocessor(remove_stopwords=True,
    ↪use_stemming=True, min_token_length=2)

# Apply transformations
df["clean_no_stem"] = prep_no_stem.transform_series(df["review_text"])
df["clean_stem"] = prep_stem.transform_series(df["review_text"])

# Vocabulary report
vocab_report = pd.DataFrame(
    {
        "raw_text": text_stats(df["review_text"]),
        "clean_no_stem": text_stats(df["clean_no_stem"]),
        "clean_stem": text_stats(df["clean_stem"]),
    }
).T.reset_index(names="view")

# Reduction metric
vocab_no_stem = vocab_report.loc[vocab_report.view == "clean_no_stem",
    ↪"vocab_size"].iloc[0]
vocab_stem = vocab_report.loc[vocab_report.view == "clean_stem", "vocab_size"].
    ↪iloc[0]
vocab_reduction_pct = 100 * (vocab_no_stem - vocab_stem) / max(vocab_no_stem, 1)

[21]: # --- Collision analysis (stemming is expected to be available) ---
if prep_stem.stemmer is None:
    raise RuntimeError("Stemming was requested but no Portuguese stemmer is
    ↪available.")

tokens = (
    df["review_text"]
    .str.lower()
    .str.findall(r"[a-zâ-öø-ÿ]+")
    .explode()
    .dropna()
)
tokens = tokens[tokens.str.len() >= 2]

unique_tokens = tokens.drop_duplicates()
stems = unique_tokens.map(prepare_stem.stemmer.stem)
mapping = pd.DataFrame({"token": unique_tokens, "stem": stems})
```

```

grouped = mapping.groupby("stem")["token"].agg(list)
collisions = grouped[grouped.str.len() > 1]
collision_ratio = len(collisions) / max(len(grouped), 1)

collision_df = (
    collisions.sort_values(key=lambda x: x.str.len(), ascending=False)
    .head(15)
    .apply(
        lambda toks: {
            "n_original_tokens": len(toks),
            "examples": ", ".join(sorted(toks)[:5]),
        }
    )
    .apply(pd.Series)
    .reset_index()
)

# --- Summary ---
summary = pd.DataFrame(
    {
        "metric": [
            "vocab_reduction_pct_stem_vs_no_stem",
            "stem_collision_ratio",
            "n_stem_collisions",
        ],
        "value": [
            round(vocab_reduction_pct, 2),
            round(collision_ratio, 4),
            int(len(collisions)),
        ],
    }
)

# --- Output ---
display(vocab_report.round(2))
display(summary)
if not collision_df.empty:
    display(collision_df)

display(df[["review_text", "clean_no_stem", "clean_stem"]].head(12))

```

	view	vocab_size	avg_tokens_per_review
0	raw_text	15061.0	11.60
1	clean_no_stem	14998.0	8.23
2	clean_stem	12196.0	8.23

metric	value
--------	-------

```

0 vocab_reduction_pct_stem_vs_no_stem    18.6800
1          stem_collision_ratio          0.1485
2          n_stem_collisions    1815.0000

```

```

      stem n_original_tokens \
0      compr                13
1      acab                 9
2      entreg               9
3      receb                9
4      atend                9
5      cont                 9
6      transport            9
7      cort                 8
8      est                  8
9      sent                 8
10     troc                 8
11     marc                 8
12     grav                 8
13     vend                 7
14     engan                7

```

```

                                examples
0      compr, comprada, compradas, comprado, comprador
1      acabada, acabadas, acabado, acabados, acabamento
2      entreg, entregada, entregado, entregador, entregadora
3      receb, recebendo, receber, recebida, recebidas
4      atendendo, atender, atendes, atendida, atendidas
5      contado, contados, contando, contar, contendo
6      transport, transportada, transportado, transportador, transportadora
7      cortada, cortadas, cortado, cortador, cortados
8      est, estado, estados, estando, estante
9      sentada, sentado, sentar, sentido, sentidos
10     trocada, trocadas, trocado, trocador, trocadores
11     marcada, marcadas, marcado, marcador, marcadores
12     gravado, gravador, gravadora, gravando, gravar
13     vend, vendendo, vender, vendida, vendidas
14     enganada, enganado, enganador, enganadores, enganados

```

```

                                review_text \
0
↳      recebi bem antes do prazo estipulado
1      parabéns lojas lannister adorei
↳ comprar pela internet seguro e prático parabéns a todos feliz páscoa
2 aparelho eficiente no site a marca do aparelho esta impresso como
↳ desinfector e ao chegar esta com outro nome atualizar com a marca corr...
3
↳      mas um pouco travando pelo valor ta boa

```

4
 ↳ vendedor confiável produto ok e entrega antes do prazo
 5
 ↳ gostaria de saber o que houve sempre recebi e essa compra agora me decepcionou
 6
 ↳ péssimo
 7
 ↳ loja nota
 8
 ↳ obrigado pela atenção amim dispensada
 9 a compra foi realizada facilmente a entrega foi efetuada muito antes do prazo
 ↳ dado o produto já começou a ser usado e até o presente sem...
 10
 ↳ relógio muito bonito e barato
 11
 ↳ não gostei comprei gato por lebre
 ↳ clean_no_stem \

0
 ↳ recebi bem antes prazo estipulado
 1
 ↳ parabéns lojas lannister adorei comprar pela internet seguro prático parabéns todos feliz páscoa
 2 aparelho eficiente site marca aparelho impresso como desinfector chegar
 ↳ outro nome atualizar marca correta vez mesmo aparelho
 3
 ↳ pouco travando pelo valor ta boa
 4
 ↳ vendedor confiável produto ok entrega antes prazo
 5
 ↳ gostaria saber houve sempre recebi essa compra agora me decepcionou
 6
 ↳ péssimo
 7
 ↳ loja nota
 8
 ↳ obrigado pela atenção amim dispensada
 9 compra realizada facilmente entrega efetuada muito antes prazo
 ↳ dado produto já começou usado até presente problemas
 10
 ↳ relógio muito bonito barato
 11
 ↳ não gostei comprei gato lebre
 ↳ clean_stem

```

0                                     recebi bem ant prazo estipul
1                                     parabén loja lannist adorei compr pela
2 internet seguro prático parabén todo feliz páscoa
3 aparelho eficiente site marca aparelho impresso como desinfector cheg outro
4 nome atualiz marca correta vez mesmo aparelho
5                                     pouco trav pelo valor ta boa
6                                     vendedor confiável produto ok entrega ant prazo
7                                     gostaria sab houve
8 sempre recebi essa compra agora me decpcionou
9                                     péssimo
10                                    loja nota
11                                    obrig pela atençao amim dispens
12                                    compra realiz facil entrega efetu muito ant prazo dado
13 produto já começou usado até presente problema
14                                    relógio muito bonito barato
15                                    não gostei comprei gato lebre

```

1.1.3 1.3 Final text preprocessing

This step locks the selected preprocessing pipeline and saves the modeling-ready review text table.

```

[22]: # Use the full cleaned dataset produced in 1.2 (no re-loading from CSV)
reviews = raw_reviews[raw_reviews["review_text"].str.len() >=
selected_min_chars].copy()

preprocessor = PortugueseTextPreprocessor(
    remove_stopwords=True,
    use_stemming=False,
    min_token_length=2,
)
reviews["clean_text"] = preprocessor.transform_series(reviews["review_text"])
reviews = reviews[reviews["clean_text"].str.len() > 0].reset_index(drop=True)

print(f"Filter threshold used (min_chars): {selected_min_chars}")
print(f"Rows after filtering + preprocessing: {len(reviews):,}")
reviews[["review_text", "clean_text"]].head(8)

```

```

Filter threshold used (min_chars): 12
Rows after filtering + preprocessing: 36,567

```

```

[22]: review_text \
0
recebi bem antes do prazo estipulado
1                                parabéns lojas lannister adorei
comprar pela internet seguro e prático parabéns a todos feliz páscoa
2 aparelho eficiente no site a marca do aparelho esta impresso como desinfector
e ao chegar esta com outro nome atualizar com a marca corr...
3
mas um pouco travando pelo valor ta boa
4
vendedor confiável produto ok e entrega antes do prazo
5                                gostaria de
saber o que houve sempre recebi e essa compra agora me decpcionou
6
obrigado pela atenção amim dispensada
7 a compra foi realizada facilmente a entrega foi efetuada muito antes do prazo
dado o produto já começou a ser usado e até o presente sem...

                                clean_text
0
recebi bem antes prazo estipulado
1                                parabéns lojas lannister adorei comprar pela
internet seguro prático parabéns todos feliz páscoa
2 aparelho eficiente site marca aparelho impresso como desinfector chegar outro
nome atualizar marca correta vez mesmo aparelho
3
pouco travando pelo valor ta boa
4
vendedor confiável produto ok entrega antes prazo
5                                gostaria saber
houve sempre recebi essa compra agora me decpcionou
6
obrigado pela atenção amim dispensada
7 compra realizada facilmente entrega efetuada muito antes prazo dado
produto já começou usado até presente problemas

```

1.1.4 1.4 Baseline Sentiment Annotation

This step creates consistent sentiment labels from ratings and lexicon cues for diagnostics and evaluation only.

```

[23]: # Make sure rating_group exists (as you already have)
reviews["rating_group"] = np.select(
    [
        reviews["review_score"] <= 2,
        reviews["review_score"] == 3,
        reviews["review_score"] >= 4,
    ],

```

```

    ["negative", "neutral", "positive"],
    default="neutral",
)

lexicon_path = LEXICON_DIR / "oplexicon_v3.0.txt"
if not lexicon_path.exists():
    raise FileNotFoundError(f"Lexicon file not found: {lexicon_path}")

print(f"Loaded lexicon source: {lexicon_path}")

```

Loaded lexicon source: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-Semantic-P15\data\lexicons\oplexicon_v3.0.txt

```

[24]: # Load lexicon into a DataFrame
      # Here we assume a tab-separated file with at least: term, polarity
      lex_df = pd.read_csv(lexicon_path, sep="\t", encoding="utf-8")

      # Inspect once to be sure of column names
      print(lex_df.columns)

      # 4) Build a simple token -> valence dict
      # Adjust 'term' and 'polarity' to your actual column names
      term_col = "term"
      pol_col = "polarity"

      lex_df[term_col] = lex_df[term_col].astype(str).str.strip().str.lower()
      lex_df = lex_df.dropna(subset=[term_col, pol_col])

      pt_valence = (
          lex_df
          .groupby(term_col)[pol_col]
          .mean()           # average if duplicates
          .to_dict()
      )

      print(f"Lexicon entries: {len(pt_valence):,}")

```

Index(['term', 'type', 'polarity', 'polarity_revision'], dtype='str')
 Lexicon entries: 31,944

```

[25]: # Compute lexicon score/group, then set rating-based primary sentiment label
      reviews["lexicon_score"] = (
          reviews["clean_text"]
          .fillna("")
          .str.lower()
          .str.split()
          .map(lambda toks: score_review(toks, pt_valence))
      )

```

```

reviews["lexicon_group"] = np.select(
    [
        reviews["lexicon_score"] > 0,
        reviews["lexicon_score"] < 0,
    ],
    ["positive", "negative"],
    default="neutral",
)

# Primary sentiment label for RQ analysis
reviews["sentiment_label"] = reviews["rating_group"]

# Diagnostic flags for label quality / semantic nuance
reviews["label_agree"] = reviews["rating_group"] == reviews["lexicon_group"]
reviews["polite_negative_flag"] = (
    (reviews["rating_group"] == "negative")
    & (reviews["lexicon_group"].isin(["neutral", "positive"])))
)
reviews["mixed_positive_flag"] = (
    (reviews["rating_group"] == "positive")
    & (reviews["lexicon_group"] == "negative")
)

cm_counts = pd.crosstab(reviews["rating_group"], reviews["lexicon_group"])
cm_norm = cm_counts.div(cm_counts.sum(axis=1), axis=0)

label_diag = pd.DataFrame(
    [
        {
            "agreement_rate": float(reviews["label_agree"].mean()),
            "polite_negative_share": float(reviews["polite_negative_flag"].
↪mean()),
            "mixed_positive_share": float(reviews["mixed_positive_flag"].
↪mean()),
            "n_reviews": int(len(reviews)),
        }
    ]
)

display(cm_counts)
display(cm_norm.round(3))
display(label_diag)

```

lexicon_group	negative	neutral	positive
rating_group			
negative	2697	5317	2563

neutral	675	1388	1185
positive	1417	8628	12697

lexicon_group	negative	neutral	positive
rating_group			
negative	0.255	0.503	0.242
neutral	0.208	0.427	0.365
positive	0.062	0.379	0.558

	agreement_rate	polite_negative_share	mixed_positive_share	n_reviews
0	0.458938	0.215495	0.038751	36567

Detailed interpretation moved to report; notebook keeps only execution outputs.

```
[26]: # Quick consistency view for rating-based primary labels vs lexical diagnostics
label_quality_by_rating = (
    reviews.groupby("rating_group")
    .agg(
        n=("review_score", "size"),
        agreement_rate=("label_agree", "mean"),
        polite_negative_rate=("polite_negative_flag", "mean"),
        mixed_positive_rate=("mixed_positive_flag", "mean"),
    )
    .reset_index()
)

display(label_quality_by_rating)
```

	rating_group	n	agreement_rate	polite_negative_rate \
0	negative	10577	0.254987	0.745013
1	neutral	3248	0.427340	0.000000
2	positive	22742	0.558306	0.000000

	mixed_positive_rate
0	0.000000
1	0.000000
2	0.062308

Detailed interpretation moved to report; notebook keeps only execution outputs.

```
[27]: # Heatmap of normalised confusion matrix
cm_norm_reset = cm_norm.reset_index().melt(
    id_vars="rating_group",
    var_name="lexicon_group",
    value_name="ratio",
)

fig_cm = px.imshow(
    cm_norm.values,
    labels=dict(x="lexicon_group", y="rating_group", color="ratio"),
```

```

x=cm_norm.columns,
y=cm_norm.index,
text_auto=".2f",
aspect="auto",
color_continuous_scale="Blues",
title="Rating vs lexicon sentiment",
)
fig_cm.show()

```

Detailed interpretation moved to report; notebook keeps only execution outputs.

```

[28]: print("NEGATIVE stars + NEGATIVE lexicon")
display(sample_examples(reviews, "negative", "negative"))

print("NEGATIVE stars + POSITIVE lexicon")
display(sample_examples(reviews, "negative", "positive"))

print("POSITIVE stars + NEGATIVE lexicon")
display(sample_examples(reviews, "positive", "negative"))

print("NEUTRAL stars + NEGATIVE lexicon")
display(sample_examples(reviews, "neutral", "negative"))

```

NEGATIVE stars + NEGATIVE lexicon

	review_score	rating_group	lexicon_group	lexicon_score	\
27	1	negative	negative	-2.0	
39	1	negative	negative	-1.0	
49	1	negative	negative	-2.0	
57	1	negative	negative	-1.0	
65	1	negative	negative	-1.0	

	review_text	\
27	produto muito inferior mal acabado	
39	comprei tres pacotes de cinco folhas cada de papel transfer para tecido escuro e so recebi dois	
49	eu não recebi o produto e consta no sistema que eu recebi além de pagar caro do frete	
57	comprei o produto dia de fevereiro e hoje dia de marco não fora entregue na minha residência não sei se os correios desse brasil e péssi...	
65	faltou produto e os que recebi veio quebrado	
	review_en	

27 very inferior poorly finished product

39 I bought three packages of five sheets each of transfer paper for dark fabric and only received two

49 I didn't receive the product and it appears in the system that I received it in addition to paying a lot of shipping.

57 I bought the product on February and today, March Day, it was not delivered to my home. I don't know if the post office in Brazil is ter...

65 The product was missing and the ones I received were broken

NEGATIVE stars + POSITIVE lexicon

	review_score	rating_group	lexicon_group	lexicon_score	\
37	1	negative	positive	1.0	
58	1	negative	positive	1.0	
76	1	negative	positive	1.0	
84	1	negative	positive	1.0	
99	2	negative	positive	2.0	

este foi o pedido balde com peças blocos de montar un r cada não foi entregue vendido e entregue targaryen tapete de eva n letras peças ...

58 fiz minha compra faz dias e não recebi ainda meu produto precisa melhorar nas entregas

76 não posso estou aguardando a chegada do produto que comprei logo que isso aconteça terei minha opinião formada

84 demora na entrega detestei o atendimento e nunca mais compro baratheon quando vendia por eles mesmos era bem melhor

99 foi marcado o dia para entrega do pedido e no rastreamento marcava que o produto já havia sido entregue na data mas na realidade não foi...

This was the order bucket with pieces building blocks un r each was not delivered sold and delivered targaryen eva rug n letters pieces ...

58 I made my purchase a few days ago and I still haven't received my product, delivery needs to improve

76 I can't, I'm waiting for the product I bought to arrive, as soon as that happens I'll have my opinion formed.

84 delay in delivery I hated the service and I will never buy baratheon again when I sold it by themselves it was much better

99 the day was set for delivery of the order and the tracking indicated that the product had already been delivered on the date but in real...

POSITIVE stars + NEGATIVE lexicon

	review_score	rating_group	lexicon_group	lexicon_score	\
6	5	positive	negative	-1.0	
11	4	positive	negative	-1.0	
21	4	positive	negative	-1.0	
22	5	positive	negative	-1.0	
33	5	positive	negative	-1.0	

↪ review_text \

6

↪ obrigado pela atencao amim dispensada

11 recebi exatamente o que esperava as demais encomendas de outros

↪ vendedores atrasaram mas esta chegou no prazo

21 ótima loja para parceria rapidíssima produtos bem embalados e de qualidade

↪ só o custo do frete que foi meio azedo

22

↪ recomendo o vendedor

33 estao de parabens sempre chega

↪ com muiiita antecedencia muito obrigada

↪ review_en

6

↪ thank you for the attention you gave me

11 I received exactly what I expected, other orders from other sellers

↪ were delayed but this one arrived on time

21 Great store for a very quick partnership, well packaged and quality

↪ products, just the shipping cost was a bit sour

22

↪ I recommend the seller

33 Congratulations always

↪ arrives well in advance, thank you very much

NEUTRAL stars + NEGATIVE lexicon

	review_score	rating_group	lexicon_group	lexicon_score	\
78	3	neutral	negative	-2.0	
128	3	neutral	negative	-2.0	
169	3	neutral	negative	-2.0	
184	3	neutral	negative	-1.0	
218	3	neutral	negative	-1.0	

↪ review_text \

78 não recomendaria esta loja nem pretendo voltar a comprar nela a mercadoria

↪ foi entregue após o prazo e quase mês após a compra no site d...

128 a embalagem deixou a desejar por

↪ pouco o produto não foi danificado a caixa estava toda amassada

169 houve alguma confusão entre vendedor e o stark pois recebi um email do
↳stark pedindo o cancelamento do produto por algum imprevisto na e...
184
↳ so recebemos um relógio pelo correio att
218 o preço foi ótimo o prazo de entrega que é longo mais de mês para entregar
↳comprei cartuchos e entregaram apenas reclamei e enviaram o o...

review_en

78 I would not recommend this store nor do I intend to buy from them again the
↳merchandise was delivered after the deadline and almost a mo...
128 the packaging left a little
↳to be desired the product was not damaged the box was all dented
169 There was some confusion between the seller and Stark because I received an
↳email from Stark asking for the product to be canceled due t...
184
↳ we only received a watch in the mail att
218 The price was great, the delivery time was long, more than a month to
↳deliver, I bought cartridges and they delivered, I just complained...

Detailed interpretation moved to report; notebook keeps only execution outputs.

Detailed interpretation moved to report; notebook keeps only execution outputs.

```
[29]: reviews.to_pickle(PREPROCESSED_PKL)

preprocessed_cache_info = pd.DataFrame([
    {
        "cache_path": str(PREPROCESSED_PKL),
        "rows": int(len(reviews)),
        "columns": int(reviews.shape[1]),
        "selected_min_chars": int(selected_min_chars),
    }
])

print(f"Saved preprocessed dataset to: {PREPROCESSED_PKL}")
display(preprocessed_cache_info)
```

Saved preprocessed dataset to: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-Semantic-P15\submission\data\processed\reviews_preprocessed.pkl

cache_path \

0 C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
↳3-4\NLP-Semantic-P15\submission\data\processed\reviews_preprocessed.pkl

	rows	columns	selected_min_chars
0	36567	16	12

1.2 2 - Embedding Engineering

This section builds comparable embedding spaces across models and validates local semantic neighborhoods.

1.2.1 2.1 Dense Vector Extraction with SemanticEmbedder

We generate aligned dense vectors for BERTimbau, MiniLM, and OpenCLIP on the same filtered review set.

```
[30]: reviews = pd.read_pickle(PREPROCESSED_PKL)
print(f"Loaded preprocessed dataset from: {PREPROCESSED_PKL}")
print(f"Rows: {len(reviews):,} | Columns: {reviews.shape[1]:,}")

model_alias = {
    "bertimbau": "bertimbau",
    "multilingual_minilm": "mbert",
    "openclip_text": "clip",
}

unknown_keys = [k for k in cfg.model_keys if k not in model_alias]
if unknown_keys:
    raise ValueError(f"Unsupported model keys in cfg.model_keys: {unknown_keys}")
if cfg.model_key not in cfg.model_keys:
    raise ValueError(f"cfg.model_key={cfg.model_key} must be one of {cfg.model_keys}")

model_run_order = list(cfg.model_keys)
selected_model_type = model_alias[cfg.model_key]

print(f"Model loop order: {model_run_order}")
print(f"Active model for Phase 3-4: {cfg.model_key} -> {selected_model_type}")

reviews[["review_score", "sentiment_label", "clean_text"]].head(5)
```

```
Loaded preprocessed dataset from: C:\Users\HP\OneDrive -
student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-
Semantic-P15\submission\data\processed\reviews_preprocessed.pkl
Rows: 36,567 | Columns: 16
Model loop order: ['bertimbau', 'multilingual_minilm', 'openclip_text']
Active model for Phase 3-4: multilingual_minilm -> mbert
```

```
[30]:   review_score sentiment_label \
0          5          positive
1          5          positive
2          4          positive
3          4          positive
4          5          positive
```

```

                                clean_text
0
recebi bem antes prazo estipulado
1                                parabéns lojas lannister adorei comprar pela
internet seguro prático parabéns todos feliz páscoa
2  aparelho eficiente site marca aparelho impresso como desinfector chegar outro
nome atualizar marca correta vez mesmo aparelho
3
pouco travando pelo valor ta boa
4
vendedor confiável produto ok entrega antes prazo

```

```

[31]: # Load model embeddings from cache (preferred) to avoid re-running heavy
      ↪ extraction
print(f"Embedding cache directory: {EMBED_DIR}")

existing_raw = globals().get("embeddings_raw_by_model", {})
existing_src = globals().get("embedding_source_by_model", {})

embeddings_raw_by_model = {}
embedding_source_by_model = {}
cache_rows = []
missing_models = []

for model_key in model_run_order:
    pattern_exact = f"{model_key}_n{len(reviews)}_d*_raw.npy"
    cached_files = sorted(EMBED_DIR.glob(pattern_exact))
    if not cached_files:
        cached_files = sorted(EMBED_DIR.glob(f"{model_key}_n*_d*_raw.npy"))

    if cached_files:
        raw_path = cached_files[0]
        matrix = np.load(raw_path).astype(np.float32)
        embeddings_raw_by_model[model_key] = matrix
        embedding_source_by_model[model_key] = f"cache:{raw_path.name}"
    elif model_key in existing_raw:
        matrix = existing_raw[model_key].astype(np.float32)
        embeddings_raw_by_model[model_key] = matrix
        embedding_source_by_model[model_key] = existing_src.get(model_key,
      ↪ "session_memory")
    else:
        missing_models.append(model_key)
        continue

    cache_rows.append(
        {

```

```

        "model_key": model_key,
        "embedding_source": embedding_source_by_model[model_key],
        "n_samples": int(matrix.shape[0]),
        "n_dims": int(matrix.shape[1]),
    }
)

if missing_models:
    print(f"Missing embeddings for: {missing_models}")
    print("Run next cell to extract missing models.")

if not embeddings_raw_by_model:
    raise RuntimeError("No embeddings loaded.")

raw_shape_df = pd.DataFrame(cache_rows).sort_values("model_key").
    ↪reset_index(drop=True)
display(raw_shape_df)

```

Embedding cache directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-Semantic-P15\data\embeddings

	model_key	embedding_source \
0	bertimbau	cache:bertimbau_n36567_d768_raw.npy
1	multilingual_minilm	cache:multilingual_minilm_n36567_d384_raw.npy
2	openclip_text	cache:openclip_text_n36567_d512_raw.npy

	n_samples	n_dims
0	36567	768
1	36567	384
2	36567	512

```

[32]: logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(levelname)s -
    ↪%(message)s")

texts = reviews["clean_text"].fillna("").astype(str).tolist()

# Keep cache-loaded matrices if present; only compute missing models.
embeddings_raw_by_model = globals().get("embeddings_raw_by_model", {}) or {}
embedding_source_by_model = globals().get("embedding_source_by_model", {}) or {}

missing_models = [m for m in model_run_order if m not in
    ↪embeddings_raw_by_model]
for model_key in missing_models:
    model_type = model_alias[model_key]
    print(f"\n[{model_key}] Extracting raw embeddings...")

    embedder = SemanticEmbedder(model_type=model_type, batch_size=32,
    ↪max_seq_length=128)

```

```

matrix = embedder.get_embeddings(texts=texts, apply_l2_norm=False)

embeddings_raw_by_model[model_key] = matrix.astype(np.float32)
embedding_source_by_model[model_key] = f"semantic_embedder:{model_type}"
print(f"[{model_key}] Raw shape: {matrix.shape}")

raw_shape_df = pd.DataFrame(
    [
        {
            "model_key": model_key,
            "embedding_source": embedding_source_by_model.get(model_key,
↪ "unknown"),
            "n_samples": int(matrix.shape[0]),
            "n_dims": int(matrix.shape[1]),
        }
        for model_key, matrix in embeddings_raw_by_model.items()
    ]
).sort_values("model_key").reset_index(drop=True)

display(raw_shape_df)

```

	model_key	embedding_source \
0	bertimbau	cache:bertimbau_n36567_d768_raw.npy
1	multilingual_minilm	cache:multilingual_minilm_n36567_d384_raw.npy
2	openclip_text	cache:openclip_text_n36567_d512_raw.npy

	n_samples	n_dims
0	36567	768
1	36567	384
2	36567	512

```

[33]: # Export embedding matrices for reuse
print(f"Embedding cache directory: {EMBED_DIR}")

if "embeddings_raw_by_model" not in globals() or len(embeddings_raw_by_model)
↪ == 0:
    raise RuntimeError("No raw embeddings found. Run cell 50 first.")

# Build L2 embeddings on the fly if cell 54 has not been run yet.
if "embeddings_l2_by_model" not in globals() or len(embeddings_l2_by_model) ==
↪ 0:
    embeddings_l2_by_model = {}
    for model_key, raw_matrix in embeddings_raw_by_model.items():
        raw_norms = np.linalg.norm(raw_matrix, axis=1)
        l2_matrix = raw_matrix / np.clip(raw_norms[:, None], 1e-12, None)
        embeddings_l2_by_model[model_key] = l2_matrix.astype(np.float32)

```

```

selected_min_chars_val = globals().get("selected_min_chars", None)

export_rows = []
missing_models = []

for model_key in model_run_order:
    if model_key not in embeddings_raw_by_model or model_key not in_
↳ embeddings_l2_by_model:
        missing_models.append(model_key)
        continue

    model_type = model_alias.get(model_key, model_key)
    source = embedding_source_by_model.get(model_key, f"semantic_embedder:
↳ {model_type}")

    raw_matrix = embeddings_raw_by_model[model_key].astype(np.float32)
    l2_matrix = embeddings_l2_by_model[model_key].astype(np.float32)

    run_tag = f"{model_key}_n{l2_matrix.shape[0]}_d{l2_matrix.shape[1]}"
    raw_npy_path = EMBED_DIR / f"{run_tag}_raw.npy"
    l2_npy_path = EMBED_DIR / f"{run_tag}_l2.npy"
    meta_json_path = EMBED_DIR / f"{run_tag}_meta.json"

    np.save(raw_npy_path, raw_matrix)
    np.save(l2_npy_path, l2_matrix)

    meta = {
        "model_key": model_key,
        "embedding_source": source,
        "semantic_model_type": model_type,
        "n_samples": int(l2_matrix.shape[0]),
        "dim": int(l2_matrix.shape[1]),
        "selected_min_chars": int(selected_min_chars_val) if_
↳ selected_min_chars_val is not None else None,
        "preprocessed_cache": str(PREPROCESSED_PKL),
        "raw_npy_path": str(raw_npy_path),
        "l2_npy_path": str(l2_npy_path),
        "meta_json_path": str(meta_json_path),
        "created_utc": pd.Timestamp.now("UTC").isoformat(),
    }
    meta_json_path.write_text(json.dumps(meta, ensure_ascii=False, indent=2),_
↳ encoding="utf-8")
    export_rows.append(meta)

if missing_models:
    print(f"Skipped models not loaded in current session: {missing_models}")

```

```

if not export_rows:
    raise RuntimeError("No embeddings were exported. Run cell 50 first.")

export_df = pd.DataFrame(export_rows).sort_values("model_key").
↳reset_index(drop=True)
display(
    export_df[
        [
            "model_key",
            "semantic_model_type",
            "n_samples",
            "dim",
            "raw_npy_path",
            "l2_npy_path",
            "meta_json_path",
        ]
    ]
)
print(f"Saved embedding artifacts to: {EMBED_DIR}")

```

Embedding cache directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\data\embeddings

	model_key	semantic_model_type	n_samples	dim	\
0	bertimbau	bertimbau	36567	768	
1	multilingual_minilm	mbert	36567	384	
2	openclip_text	clip	36567	512	

```

↳
raw_npy_path \
0 C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
↳3-4\NLP-Semantic-P15\data\embeddings\bertimbau_n36567_d768_raw.npy
1 C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
↳3-4\NLP-Semantic-P15\data\embeddings\multilingual_minilm_n36567_d384_raw.npy
2 C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
↳3-4\NLP-Semantic-P15\data\embeddings\openclip_text_n36567_d512_raw.npy

```

```

↳
l2_npy_path \
0 C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
↳3-4\NLP-Semantic-P15\data\embeddings\bertimbau_n36567_d768_l2.npy
1 C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
↳3-4\NLP-Semantic-P15\data\embeddings\multilingual_minilm_n36567_d384_l2.npy
2 C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
↳3-4\NLP-Semantic-P15\data\embeddings\openclip_text_n36567_d512_l2.npy

```

```

    meta_json_path
0      C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
    3-4\NLP-Semantic-P15\data\embeddings\bertimbau_n36567_d768_meta.json
1      C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
    3-4\NLP-Semantic-P15\data\embeddings\multilingual_minilm_n36567_d384_meta.json
2      C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
    3-4\NLP-Semantic-P15\data\embeddings\openclip_text_n36567_d512_meta.json

Saved embedding artifacts to: C:\Users\HP\OneDrive -
student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-Semantic-P15\data\embeddings

```

1.2.2 2.2 Normalisation and distance metric

We apply L2 normalization to ensure cosine-based comparisons are consistent across models.

```

[34]: # L2-normalize embeddings for cosine-based geometry (all models)
embeddings_l2_by_model: dict[str, np.ndarray] = {}
norm_rows = []

for model_key, matrix in embeddings_raw_by_model.items():
    raw_norms = np.linalg.norm(matrix, axis=1)
    matrix_l2 = matrix / np.clip(raw_norms[:, None], 1e-12, None)
    l2_norms = np.linalg.norm(matrix_l2, axis=1)

    embeddings_l2_by_model[model_key] = matrix_l2.astype(np.float32)

    norm_rows.append(
        {
            "model_key": model_key,
            "raw_norm_mean": float(raw_norms.mean()),
            "raw_norm_std": float(raw_norms.std()),
            "raw_norm_min": float(raw_norms.min()),
            "raw_norm_max": float(raw_norms.max()),
            "norm_after_l2_mean": float(l2_norms.mean()),
            "norm_after_l2_std": float(l2_norms.std()),
            "n_samples": int(matrix.shape[0]),
            "n_dims": int(matrix.shape[1]),
        }
    )

embed_stats = pd.DataFrame(norm_rows).sort_values("model_key").
    reset_index(drop=True)
display(embed_stats)

embeddings = embeddings_l2_by_model[cfg.model_key]
print(f"Active normalized embedding shape: {embeddings.shape}")

```

	model_key	raw_norm_mean	raw_norm_std	raw_norm_min	\
0	bertimbau	7.678200	0.636176	5.682571	
1	multilingual_minilm	3.243995	0.463427	1.817343	
2	openclip_text	10.481853	1.023574	6.165544	

	raw_norm_max	norm_after_l2_mean	norm_after_l2_std	n_samples	n_dims
0	15.492994	1.0	3.543649e-08	36567	768
1	5.920916	1.0	3.518197e-08	36567	384
2	13.056970	1.0	5.331329e-08	36567	512

Active normalized embedding shape: (36567, 384)

1.2.3 2.3 k-NN analysis

We run nearest-neighbor checks to validate local semantic coherence before clustering and projection.

```
[35]: rng = np.random.default_rng(cfg.random_state)
```

```
[36]: for idx in rng.integers(0, len(reviews), size=3):
    neighbor_df, anchor_sent = inspect_neighbors(
        anchor_idx=int(idx),
        reviews_df=reviews,
        embeddings_l2_by_model=embeddings_l2_by_model,
        top_k=5,
    )
    if neighbor_df.empty:
        print("Skipping anchor: neighbor table is empty.")
        continue

    summarize_neighbors(neighbor_df, anchor_sent)
    plot_local_neighbourhood(
        anchor_idx=int(idx),
        active_matrix=embeddings_l2_by_model[cfg.model_key],
        reviews_df=reviews,
        model_key=cfg.model_key,
        random_state=cfg.random_state,
        top_k=20,
    )
```

```
=====
=====
Anchor idx: 3263 | sentiment: positive
PT: recebi o produto bem antes do prazo para entrega informado no ato da compra
ameiiii
EN: I received the product well before the delivery date stated at the time of
purchase, I loved it.
=====
=====
```

	model_key	rank	cosine_similarity	sentiment	same_sentiment \
0	bertimbau	1	0.940357	positive	True
1	bertimbau	2	0.930035	positive	True
2	bertimbau	3	0.928207	positive	True
3	bertimbau	4	0.928187	positive	True
4	bertimbau	5	0.926438	positive	True
5	multilingual_minilm	1	0.890643	positive	True
6	multilingual_minilm	2	0.890643	positive	True
7	multilingual_minilm	3	0.890643	positive	True
8	multilingual_minilm	4	0.890643	positive	True
9	multilingual_minilm	5	0.885786	positive	True
10	openclip_text	1	0.987195	positive	True
11	openclip_text	2	0.985275	positive	True
12	openclip_text	3	0.984130	positive	True
13	openclip_text	4	0.984080	positive	True
14	openclip_text	5	0.983963	positive	True

	snippet \	
0		amei o
	produto chegou bem antes do prazo de entrega	
1		amei o
	produto entrega bem antes da data prevista	
2		adorei o produto e
	foi entregue bem antes da data marcada	
3		recebi o produto bem
	antes da data prevista adorei essa compra	
4		minha encomenda
	chegou bem antes do prazo de entrega amei	
5		
	recebi o produto antes do prazo	
6		
	recebi o produto antes do prazo	
7		
	recebi o produto antes do prazo	
8		
	recebi o produto antes do prazo	
9		recebi
	o produto bem antes do prazo informado	
10		produto veio
	certinho entrega até antes do prazo amei	
11	prezados recebi o produto comprado em perfeito estado e dentro do prazo	
	recomendo esta empresa att george carvalho	
12		recebi o produto correto com
	prazo menor ao informado no ato da compra	
13		recebi ate antes do prazo previsto e de acordo com o
	produto comprado obrigado e deus abençoe ok	

14 recibi o produto antes do prazo veio conforme o pedido e gosdei. 
 ↳ muito parabens pela seriedade obrigado

	snippet_en	
0		
1	I loved the product arrived well before the delivery date	
2	loved the product, delivery well before the scheduled date	I loved the
3	product and it was delivered well before the scheduled date	
4	product well before the expected date, I loved this purchase	I received the
5	My order arrived well before the delivery date, I loved it	
6	I received the product before the deadline	
7	I received the product before the deadline	
8	I received the product before the deadline	
9	I received the product before the deadline	
10	I received the product well before the stated deadline	
11	Product arrived exactly before the deadline, I loved it	
12	Dear all, I received the product purchased in perfect condition and within the deadline I recommend this company att george oak	
13	I received the correct product within a shorter period of time than stated at the time of purchase	
14	I received it before the expected deadline and in accordance with the product purchased, thank you and God bless ok	
15	I received the product before the deadline, it came as ordered and I really liked it, congratulations for your seriousness, thank you	

	model_key	share_same_sentiment
0	bertimbau	1.0
1	multilingual_minilm	1.0
2	openclip_text	1.0

Anchor idx: 28301 | sentiment: positive
PT: o produto chegou no prazo mas recusei no ato da entrega por motivos de desistência
EN: The product arrived on time but I refused upon delivery for reasons of

withdrawal

=====

	model_key	rank	cosine_similarity	sentiment	same_sentiment	\
0	bertimbau	1	0.884387	positive	True	
1	bertimbau	2	0.883224	negative	False	
2	bertimbau	3	0.875189	negative	False	
3	bertimbau	4	0.869320	positive	True	
4	bertimbau	5	0.869320	positive	True	
5	multilingual_minilm	1	0.801798	neutral	False	
6	multilingual_minilm	2	0.796254	negative	False	
7	multilingual_minilm	3	0.792560	negative	False	
8	multilingual_minilm	4	0.785802	positive	True	
9	multilingual_minilm	5	0.780733	negative	False	
10	openclip_text	1	0.996140	neutral	False	
11	openclip_text	2	0.995222	positive	True	
12	openclip_text	3	0.994920	negative	False	
13	openclip_text	4	0.994643	positive	True	
14	openclip_text	5	0.994601	negative	False	

↪ snippet \

0 recebi o produto

↪ antes do prazo de entrega e sem intercorrências

1 realizei a compra e o pagamento o prazo venceu e nada do produto ser

↪ entregue na verdade não foi nem enviado ainda péssimo

2 antes da entrega havia cancelado a

↪ compra mas mesmo assim o produto foi entregue

3

↪ recebi antes do prazo de entrega

4

↪ recebi antes do prazo de entrega

5 foi entregue dentro do prazo

↪ porém o produto veio com uma parte descosturada

6 entregaram os produtos mas

↪ não cumpriram o prazo de entrega passou do prazo

7

↪ o produto chegou avariado e recusei a entrega

8

↪ produto entregue bem antes do prazo e sem defeitos

9 até o momento o

↪ produto não chegou está fora do prazo de entrega

10 produto chegou no prazo

↪ mas com defeito estou pedindo troca ou reembolso

11 não tenho do que reclamar recebi no prazo

↪ estipulado e o produto foi o que eu havia pedido

12 o produto não foi entregue nem
 ↳ me deram satisfação após a reclamação da demora

13 o produto chegou muito antes do previsto fato louvável
 ↳ no entanto veio com defeito como devo proceder

14 até o momento o produto não
 ↳ chegou o prazo de entrega já excedeu quero posição

↳ snippet_en

0 I received the
 ↳ product before the delivery date and without any complications

1 I made the purchase and payment, the deadline expired and the product was
 ↳ not delivered, in fact it was not even sent, still terrible

2 I had canceled the
 ↳ purchase before delivery but the product was still delivered

3 I received it before the delivery date

4 I received it before the delivery date

5 It was
 ↳ delivered on time but the product came with a part unsewn

6 delivered the products but did not meet the delivery deadline.

7 The product arrived damaged and I refused delivery

8 Product delivered well ahead of schedule and without defects

9 The
 ↳ product has not yet arrived and is beyond the delivery date

10 Product arrived on time
 ↳ but was defective, I am asking for an exchange or refund

11 I have nothing to complain about, I received it
 ↳ within the stipulated time and the product was what I ordered

12 The product was not delivered
 ↳ and I was not satisfied after complaining about the delay

13 The product arrived much earlier than expected, a
 ↳ commendable fact, however it was defective, what should I do?

14 so far the product has not
 ↳ arrived the delivery time has already exceeded my position

	model_key	share_same_sentiment
0	bertimbau	0.6
1	multilingual_minilm	0.2
2	openclip_text	0.4

=====

=====

Anchor idx: 23935 | sentiment: positive

PT: chegou antes do prazo bem embalado e como esperava

EN: arrived ahead of schedule, well packaged and as expected

=====

	model_key	rank	cosine_similarity	sentiment	same_sentiment	\
0	bertimbau	1	0.943473	positive	True	
1	bertimbau	2	0.936511	positive	True	
2	bertimbau	3	0.925455	positive	True	
3	bertimbau	4	0.919605	positive	True	
4	bertimbau	5	0.917534	positive	True	
5	multilingual_minilm	1	0.921975	positive	True	
6	multilingual_minilm	2	0.916488	positive	True	
7	multilingual_minilm	3	0.912748	positive	True	
8	multilingual_minilm	4	0.911128	positive	True	
9	multilingual_minilm	5	0.908721	positive	True	
10	openclip_text	1	0.998661	positive	True	
11	openclip_text	2	0.998617	positive	True	
12	openclip_text	3	0.998397	positive	True	
13	openclip_text	4	0.997977	positive	True	
14	openclip_text	5	0.997917	positive	True	

	snippet	\
0	chegou antes do prazo bem embalado e exatamente como descrito	
1	chegou bem embalado e antes do prazo previsto	
2	chegou dentro do prazo e bem embalado	
3	chegou até antes do prazo e bem embalado	
4	chegou antes do prazo muito bem embalado	
5	chegou antes do prazo muito bem embalado	
6	chegou até antes do prazo e bem embalado	
7	chegou bem embalado e antes do prazo previsto	
8	entregou bem antes da data prevista	
9	chegou bem antes do prazo previsto e veio tudo certinho	
10	chegou até antes do prazo e bem embalado	
11	chegou antes do prazo muito bem embalado	
12	chegou antes do prazo e embalado corretamente	
13	chegou dentro do prazo e bem embalado	
14	chegou antes do prazo estipulado e bem embalado	

	snippet_en
0	Arrived ahead of schedule, well packaged and exactly as described
1	arrived well packaged and ahead of schedule
2	arrived on time and well packaged
3	arrived ahead of schedule and well packaged
4	arrived ahead of schedule very well packaged
5	arrived ahead of schedule very well packaged
6	arrived ahead of schedule and well packaged

```

7             arrived well packaged and ahead of schedule
8             Delivered well before expected date
9     It arrived well ahead of schedule and everything was perfect.
10            arrived ahead of schedule and well packaged
11            arrived ahead of schedule very well packaged
12            arrived ahead of schedule and packaged correctly
13            arrived on time and well packaged
14            arrived before the stipulated deadline and well packaged

    model_key  share_same_sentiment
0      bertimbau          1.0
1  multilingual_minilm          1.0
2      openclip_text          1.0

```

1.3 3 - Dimensionality reduction and emergent clustering

This section estimates latent structure and evaluates whether negative sentiment forms concentrated islands or diffuse overlays.

```

[37]: # --- Phase 3 artifact loading (cached outputs) ---
# Load Phase 3 artifacts from folder so clustering does not need to be re-run
PHASE3_DIR = DATA_DIR / "phase3_multi"
print(f"Embedding cache directory: {EMBED_DIR}")

if not PHASE3_DIR.exists():
    raise FileNotFoundError(f"Phase 3 folder not found: {PHASE3_DIR}")

if "model_run_order" not in globals() or not model_run_order:
    model_run_order = list(cfg.model_keys)

# Ensure reviews table is available for topic proxy and downstream evaluation.
if "reviews" not in globals() or not isinstance(reviews, pd.DataFrame) or
↳ reviews.empty:
    if not PREPROCESSED_PKL.exists():
        raise NameError(
            "`reviews` is not defined and preprocessed cache is missing. "
            "Run preprocessing cells first."
        )
    reviews = pd.read_pickle(PREPROCESSED_PKL)
    print(f"Loaded reviews from cache: {PREPROCESSED_PKL}")

if "clean_text" not in reviews.columns:
    raise ValueError(
        "`reviews` exists but `clean_text` is missing. "
        "Run preprocessing (cell 31) and sentiment labeling before this phase."
    )

reviews = reviews.reset_index(drop=True).copy()

```

```

n_rows_hint = len(reviews)
embeddings_l2_by_model = {}
embedding_source_by_model = {}
phase3_by_model = {}

kmeans_scores_all = []
purity_all = []
stability_runs_all = []
stability_summary_all = []
cluster_size_all = []
phase3_rows = []

```

Embedding cache directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-Semantic-P15\data\embeddings

```

[38]: # --- Load per-model arrays/tables from disk ---
for model_key in model_run_order:
    model_dir = PHASE3_DIR / model_key
    if not model_dir.exists():
        raise FileNotFoundError(f"Missing model folder: {model_dir}")

    X_model, emb_path = load_cached_l2(EMBED_DIR, model_key,
    ↪n_rows_hint=n_rows_hint)
    if X_model is None:
        raise FileNotFoundError(f"No cached L2 embeddings found for model:
    ↪{model_key}")

    embeddings_l2_by_model[model_key] = X_model
    embedding_source_by_model[model_key] = f"cache:{emb_path.name}"

    labels_model = np.load(model_dir / "labels_kmeans.npy").astype(int)
    spectral_labels_model = np.load(model_dir / "labels_spectral.npy").
    ↪astype(int)

    kmeans_scores_model = read_csv_or_empty(model_dir / "kmeans_scores.csv")
    stability_runs_model = read_csv_or_empty(model_dir / "stability_runs.csv")
    stability_summary_model = read_csv_or_empty(model_dir / "stability_summary.
    ↪csv")
    cluster_size_df_model = read_csv_or_empty(model_dir / "cluster_sizes.csv")
    purity_table_model = read_csv_or_empty(model_dir / "purity_table.csv")

    kmeans_final_df = read_csv_or_empty(model_dir / "kmeans_final_metrics.csv")
    spectral_df = read_csv_or_empty(model_dir / "spectral_metrics.csv")

    kmeans_final_metrics_model = kmeans_final_df.iloc[0].to_dict() if not
    ↪kmeans_final_df.empty else {}

```

```

        "model_key": model_key,
        "algorithm": "kmeans",
        "k": int(np.nan),
        "silhouette_cosine": np.nan,
        "calinski_harabasz": np.nan,
        "davies_bouldin": np.nan,
        "inertia": np.nan,
    }

    spectral_metrics_model = spectral_df.iloc[0].to_dict() if not spectral_df.
    ↪empty else {
        "model_key": model_key,
        "algorithm": "spectral",
        "k": int(np.nan),
        "silhouette_cosine": np.nan,
        "calinski_harabasz": np.nan,
        "davies_bouldin": np.nan,
    }

    manifest_path = model_dir / "phase3_manifest.json"
    if manifest_path.exists():
        best_k_model = int(json.loads(manifest_path.
    ↪read_text(encoding="utf-8")).get("best_k", 0))
    else:
        best_k_model = int(kmeans_final_metrics_model.get("k", 0) or 0)

    phase3_by_model[model_key] = {
        "embeddings": X_model,
        "best_k": best_k_model,
        "labels": labels_model,
        "spectral_labels": spectral_labels_model,
        "kmeans_scores": kmeans_scores_model,
        "purity_table": purity_table_model,
        "stability_runs": stability_runs_model,
        "stability_summary": stability_summary_model,
        "cluster_size_df": cluster_size_df_model,
        "kmeans_final_metrics": kmeans_final_metrics_model,
        "spectral_metrics": spectral_metrics_model,
    }

    if not kmeans_scores_model.empty:
        kmeans_scores_all.append(kmeans_scores_model)
    if not purity_table_model.empty:
        purity_all.append(purity_table_model)
    if not stability_runs_model.empty:
        stability_runs_all.append(stability_runs_model)
    if not stability_summary_model.empty:

```

```

        stability_summary_all.append(stability_summary_model)
    if not cluster_size_df_model.empty:
        cluster_size_all.append(cluster_size_df_model)

    kmeans_purity_row = purity_table_model[purity_table_model["algorithm"] == "kmeans"]
    if not purity_table_model.empty else pd.DataFrame()
    kmeans_purity_topic = float(kmeans_purity_row["purity_topic_proxy"].iloc[0])
    if not kmeans_purity_row.empty else np.nan
    kmeans_purity_sent = float(kmeans_purity_row["purity_sentiment"].iloc[0])
    if not kmeans_purity_row.empty else np.nan

    phase3_rows.append(
        {
            "model_key": model_key,
            "best_k": int(best_k_model),
            "kmeans_silhouette": float(kmeans_final_metrics_model.get("silhouette_cosine", np.nan)),
            "spectral_silhouette": float(spectral_metrics_model.get("silhouette_cosine", np.nan)),
            "kmeans_purity_topic_proxy": kmeans_purity_topic,
            "kmeans_purity_sentiment": kmeans_purity_sent,
        }
    )

```

```

[39]: # --- Build combined cross-model tables for quick comparison ---
# Combined tables
phase3_model_comparison = pd.DataFrame(phase3_rows).sort_values("model_key").reset_index(drop=True)
kmeans_scores_combined = pd.concat(kmeans_scores_all, ignore_index=True) if kmeans_scores_all else pd.DataFrame()
purity_table_combined = pd.concat(purity_all, ignore_index=True) if purity_all else pd.DataFrame()
stability_runs_combined = pd.concat(stability_runs_all, ignore_index=True) if stability_runs_all else pd.DataFrame()
stability_summary_combined = pd.concat(stability_summary_all, ignore_index=True) if stability_summary_all else pd.DataFrame()
cluster_size_combined = pd.concat(cluster_size_all, ignore_index=True) if cluster_size_all else pd.DataFrame()

# --- Set active-model aliases used by later cells ---
# Active model aliases for downstream cells
active3 = phase3_by_model[cfg.model_key]
embedding_source = embedding_source_by_model[cfg.model_key]
embeddings = active3["embeddings"]
X = embeddings
best_k = active3["best_k"]

```

```

labels = active3["labels"]
spectral_labels = active3["spectral_labels"]
kmeans_scores = active3["kmeans_scores"]
purity_table = active3["purity_table"]
stability_runs = active3["stability_runs"]
stability_summary = active3["stability_summary"]
cluster_size_df = active3["cluster_size_df"]
kmeans_final_metrics = active3["kmeans_final_metrics"]
spectral_metrics = active3["spectral_metrics"]

# Topic proxy used by projection and evaluation cells
text_for_topic = reviews["clean_text"].fillna("").str.lower()
topic_proxy = np.select(
    [
        text_for_topic.str.
        ↪contains(r"\b(entrega|prazo|atras|frete|transport)\w*", regex=True),
        text_for_topic.str.
        ↪contains(r"\b(produit|qualidad|defeit|quebrad|avari)\w*", regex=True),
        text_for_topic.str.
        ↪contains(r"\b(atendiment|suport|sac|respost|contat)\w*", regex=True),
        text_for_topic.str.contains(r"\b(troc|devolu|reembols|cancel)\w*",
        ↪regex=True),
        text_for_topic.str.contains(r"\b(preco|valor|car[oa]|barat|cust)\w*",
        ↪regex=True),
    ],
    ["delivery", "product", "support", "refund", "price"],
    default="other",
)

projector = ProjectionEngine(random_state=cfg.random_state)
print(f"Loaded Phase 3 artifacts from: {PHASE3_DIR}")
print(f"Models loaded: {model_run_order}")
print(f"Active model: {cfg.model_key} | best_k={best_k}")
display(phase3_model_comparison)

```

```

Loaded Phase 3 artifacts from: C:\Users\HP\OneDrive -
student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-Semantic-P15\submission\data\phase3_multi
Models loaded: ['bertimbau', 'multilingual_minilm', 'openclip_text']
Active model: multilingual_minilm | best_k=7

```

	model_key	best_k	kmeans_silhouette	spectral_silhouette \
0	bertimbau	7	0.385390	NaN
1	multilingual_minilm	7	0.099294	-0.202394
2	openclip_text	7	0.216980	NaN

	kmeans_purity_topic_proxy	kmeans_purity_sentiment
0	0.865062	0.761963
1	0.562584	0.791725

1.3.1 3.1 K-curve diagnostics and evaluator setup

We evaluate clustering quality across candidate k values and bind the selected artifacts for the next steps.

```
[40]: logging.basicConfig(level=logging.INFO, format="%(asctime)s - %(message)s")
geom_eval = GeometryClusteringEvaluator(random_state=cfg.random_state)
print("GeometryClusteringEvaluator loaded from src.notebook_phase3_helpers")
```

GeometryClusteringEvaluator loaded from src.notebook_phase3_helpers

```
[41]: # Diagnostic-only K sweep from saved embedding files
print(f"Embedding cache directory: {EMBED_DIR}")

if "model_run_order" not in globals() or not model_run_order:
    model_run_order = list(cfg.model_keys)

if "geom_eval" not in globals() or geom_eval is None:
    geom_eval = GeometryClusteringEvaluator(random_state=cfg.random_state)

# Controls for quick rerun diagnostics
DIAG_SAMPLE_FRAC = 0.35 # set 1.0 for full-data diagnostics
DIAG_MAX_ROWS = 15000 # cap per model for speed

n_rows_hint = len(reviews) if "reviews" in globals() else None
sentiment_labels = reviews["sentiment_label"].astype(str).to_numpy() if "reviews" in globals() else np.array([])

kmeans_scores_all = []
diag_rows = []
embeddings_l2_by_model = {}

for model_key in model_run_order:
    X_full, cache_path = load_cached_l2(EMBED_DIR, model_key, n_rows_hint=n_rows_hint)
    if X_full is None:
        raise FileNotFoundError(
            f"No cached L2 embedding file for model '{model_key}' under {EMBED_DIR}. "
            "Run Phase 2 export cell once, then rerun this diagnostic cell."
        )

    embeddings_l2_by_model[model_key] = X_full
    n_total = len(X_full)

    target_n = n_total
```

```

if DIAG_SAMPLE_FRAC < 1.0:
    target_n = min(target_n, max(1000, int(n_total * DIAG_SAMPLE_FRAC)))
if DIAG_MAX_ROWS is not None:
    target_n = min(target_n, int(DIAG_MAX_ROWS))

diag_idx = stratified_sentiment_indices(
    sentiment_labels=sentiment_labels,
    n_rows=n_total,
    sample_n=target_n,
    seed=cfg.random_state,
)
X_diag = X_full[diag_idx]
sample_mode = "full" if len(diag_idx) == n_total else "stratified_sentiment"

scores = run_k_grid_diagnostics(
    X_diag=X_diag,
    k_values=tuple(cfg.k_grid),
    random_state=cfg.random_state,
    geom_eval=geom_eval,
    silhouette_sample_size=getattr(geom_eval, "silhouette_sample_size",
↪None),
)
scores["model_key"] = model_key
scores["n_rows_diag"] = int(len(X_diag))
scores["n_rows_total"] = int(n_total)
scores["sample_mode"] = sample_mode
scores["cache_file"] = cache_path.name if cache_path is not None else ""

kmeans_scores_all.append(scores)
diag_rows.append(
    {
        "model_key": model_key,
        "n_rows_total": int(n_total),
        "n_rows_diag": int(len(X_diag)),
        "sample_mode": sample_mode,
        "cache_file": cache_path.name if cache_path is not None else "",
    }
)

kmeans_scores_combined = pd.concat(kmeans_scores_all, ignore_index=True)
kmeans_scores = kmeans_scores_combined.loc[kmeans_scores_combined["model_key"]
↪== cfg.model_key].reset_index(drop=True)
diag_summary = pd.DataFrame(diag_rows).sort_values("model_key").
↪reset_index(drop=True)

display(diag_summary)
display(kmeans_scores_combined)

```

Embedding cache directory: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\data\embeddings

	model_key	n_rows_total	n_rows_diag	sample_mode	\
0	bertimbau	36567	12798	stratified_sentiment	
1	multilingual_minilm	36567	12798	stratified_sentiment	
2	openclip_text	36567	12798	stratified_sentiment	

	cache_file
0	bertimbau_n36567_d768_l2.npy
1	multilingual_minilm_n36567_d384_l2.npy
2	openclip_text_n36567_d512_l2.npy

	k	silhouette_cosine	calinski_harabasz	davies_bouldin	inertia	\
0	2	0.135529	1014.483070	3.498984	4587.668945	
1	4	0.099236	774.756000	2.882053	4190.166016	
2	6	0.066254	596.715703	3.310812	4014.948242	
3	8	0.067916	489.623958	3.301877	3904.961914	
4	10	0.059673	416.181102	3.497297	3829.667480	
5	12	0.053903	363.884226	3.640232	3770.889160	
6	2	0.197650	1506.307452	2.877292	7675.763184	
7	4	0.136331	947.923077	2.684395	7019.155762	
8	6	0.107957	746.160844	3.119532	6642.141602	
9	8	0.094662	618.962395	3.012959	6408.416016	
10	10	0.101071	541.266177	2.976852	6212.697266	
11	12	0.105025	483.127531	2.917612	6060.380859	
12	2	0.264970	1446.148079	2.838650	599.013977	
13	4	0.086815	909.984388	3.132826	549.467957	
14	6	0.106646	691.330233	3.038090	524.879395	
15	8	0.057464	572.080428	2.894030	507.738617	
16	10	0.067536	507.404663	2.899827	491.275696	
17	12	0.039983	453.641013	3.071883	479.554260	

	model_key	n_rows_diag	n_rows_total	sample_mode	\
0	bertimbau	12798	36567	stratified_sentiment	
1	bertimbau	12798	36567	stratified_sentiment	
2	bertimbau	12798	36567	stratified_sentiment	
3	bertimbau	12798	36567	stratified_sentiment	
4	bertimbau	12798	36567	stratified_sentiment	
5	bertimbau	12798	36567	stratified_sentiment	
6	multilingual_minilm	12798	36567	stratified_sentiment	
7	multilingual_minilm	12798	36567	stratified_sentiment	
8	multilingual_minilm	12798	36567	stratified_sentiment	
9	multilingual_minilm	12798	36567	stratified_sentiment	
10	multilingual_minilm	12798	36567	stratified_sentiment	
11	multilingual_minilm	12798	36567	stratified_sentiment	
12	openclip_text	12798	36567	stratified_sentiment	
13	openclip_text	12798	36567	stratified_sentiment	
14	openclip_text	12798	36567	stratified_sentiment	

15	openclip_text	12798	36567	stratified_sentiment
16	openclip_text	12798	36567	stratified_sentiment
17	openclip_text	12798	36567	stratified_sentiment

	cache_file
0	bertimbau_n36567_d768_l2.npy
1	bertimbau_n36567_d768_l2.npy
2	bertimbau_n36567_d768_l2.npy
3	bertimbau_n36567_d768_l2.npy
4	bertimbau_n36567_d768_l2.npy
5	bertimbau_n36567_d768_l2.npy
6	multilingual_minilm_n36567_d384_l2.npy
7	multilingual_minilm_n36567_d384_l2.npy
8	multilingual_minilm_n36567_d384_l2.npy
9	multilingual_minilm_n36567_d384_l2.npy
10	multilingual_minilm_n36567_d384_l2.npy
11	multilingual_minilm_n36567_d384_l2.npy
12	openclip_text_n36567_d512_l2.npy
13	openclip_text_n36567_d512_l2.npy
14	openclip_text_n36567_d512_l2.npy
15	openclip_text_n36567_d512_l2.npy
16	openclip_text_n36567_d512_l2.npy
17	openclip_text_n36567_d512_l2.npy

```
[42]: display(kmeans_scores_combined)
```

	k	silhouette_cosine	calinski_harabasz	davies_bouldin	inertia	\
0	2	0.135529	1014.483070	3.498984	4587.668945	
1	4	0.099236	774.756000	2.882053	4190.166016	
2	6	0.066254	596.715703	3.310812	4014.948242	
3	8	0.067916	489.623958	3.301877	3904.961914	
4	10	0.059673	416.181102	3.497297	3829.667480	
5	12	0.053903	363.884226	3.640232	3770.889160	
6	2	0.197650	1506.307452	2.877292	7675.763184	
7	4	0.136331	947.923077	2.684395	7019.155762	
8	6	0.107957	746.160844	3.119532	6642.141602	
9	8	0.094662	618.962395	3.012959	6408.416016	
10	10	0.101071	541.266177	2.976852	6212.697266	
11	12	0.105025	483.127531	2.917612	6060.380859	
12	2	0.264970	1446.148079	2.838650	599.013977	
13	4	0.086815	909.984388	3.132826	549.467957	
14	6	0.106646	691.330233	3.038090	524.879395	
15	8	0.057464	572.080428	2.894030	507.738617	
16	10	0.067536	507.404663	2.899827	491.275696	
17	12	0.039983	453.641013	3.071883	479.554260	

	model_key	n_rows_diag	n_rows_total	sample_mode	\
0	bertimbau	12798	36567	stratified_sentiment	

1	bertimbau	12798	36567	stratified_sentiment
2	bertimbau	12798	36567	stratified_sentiment
3	bertimbau	12798	36567	stratified_sentiment
4	bertimbau	12798	36567	stratified_sentiment
5	bertimbau	12798	36567	stratified_sentiment
6	multilingual_minilm	12798	36567	stratified_sentiment
7	multilingual_minilm	12798	36567	stratified_sentiment
8	multilingual_minilm	12798	36567	stratified_sentiment
9	multilingual_minilm	12798	36567	stratified_sentiment
10	multilingual_minilm	12798	36567	stratified_sentiment
11	multilingual_minilm	12798	36567	stratified_sentiment
12	openclip_text	12798	36567	stratified_sentiment
13	openclip_text	12798	36567	stratified_sentiment
14	openclip_text	12798	36567	stratified_sentiment
15	openclip_text	12798	36567	stratified_sentiment
16	openclip_text	12798	36567	stratified_sentiment
17	openclip_text	12798	36567	stratified_sentiment

	cache_file
0	bertimbau_n36567_d768_l2.npy
1	bertimbau_n36567_d768_l2.npy
2	bertimbau_n36567_d768_l2.npy
3	bertimbau_n36567_d768_l2.npy
4	bertimbau_n36567_d768_l2.npy
5	bertimbau_n36567_d768_l2.npy
6	multilingual_minilm_n36567_d384_l2.npy
7	multilingual_minilm_n36567_d384_l2.npy
8	multilingual_minilm_n36567_d384_l2.npy
9	multilingual_minilm_n36567_d384_l2.npy
10	multilingual_minilm_n36567_d384_l2.npy
11	multilingual_minilm_n36567_d384_l2.npy
12	openclip_text_n36567_d512_l2.npy
13	openclip_text_n36567_d512_l2.npy
14	openclip_text_n36567_d512_l2.npy
15	openclip_text_n36567_d512_l2.npy
16	openclip_text_n36567_d512_l2.npy
17	openclip_text_n36567_d512_l2.npy

```
[43]: #k curve plot for silhouette score
fig_k_curve = px.line(
    kmeans_scores_combined,
    x="k",
    y="silhouette_cosine",
    color="model_key",
    markers=True,
    title=f"K-means silhouette score curve (diagnostic sample:
↳{DIAG_SAMPLE_FRAC:.0%} {DIAG_MAX_ROWS} rows)",
```

```

)
fig_k_curve.update_layout(xaxis_title="Number of clusters (k)",
    ↪yaxis_title="Silhouette Score (cosine)")
fig_k_curve.show()
#k curve plot for davies_bouldin score
fig_db_curve = px.line(
    kmeans_scores_combined,
    x="k",
    y="davies_bouldin",
    color="model_key",
    markers=True,
    title=f"K-means Davies-Bouldin score curve (diagnostic sample:
    ↪{DIAG_SAMPLE_FRAC:.0%} {DIAG_MAX_ROWS} rows)",
)
fig_db_curve.update_layout(xaxis_title="Number of clusters (k)",
    ↪yaxis_title="Davies-Bouldin Score")
fig_db_curve.show()
#k curve plot for calinski_harabasz score
fig_ch_curve = px.line(
    kmeans_scores_combined,
    x="k",
    y="calinski_harabasz",
    color="model_key",
    markers=True,
    title=f"K-means Calinski-Harabasz score curve (diagnostic sample:
    ↪{DIAG_SAMPLE_FRAC:.0%} {DIAG_MAX_ROWS} rows)",
)
fig_ch_curve.update_layout(xaxis_title="Number of clusters (k)",
    ↪yaxis_title="Calinski-Harabasz Score")
fig_ch_curve.show()

```

```

[44]: projector = ProjectionEngine(random_state=cfg.random_state)

# --- Clustering control flags ---
# Recompute Phase 3 only when needed; default uses preloaded artifacts from
    ↪cell 64.
PHASE3_RECOMPUTE = False
PHASE3_MODELS_THIS_RUN = [cfg.model_key]
PHASE3_RESET = False
PHASE3_SILHOUETTE_SAMPLE_SIZE = 12000

# K-selection controls.
# Modes: "auto_ranked" (best-practice from analysis-v3), "fixed", "per_model"
PHASE3_K_MODE = "fixed"
PHASE3_K_FIXED = 7
PHASE3_K_BY_MODEL = {
    "bertimbau": 7,

```

```

    "multilingual_minilm": 7,
    "openclip_text": 7,
}

# analysis-v3 style diagnostics: explicit k grid + stratified sample
PHASE3_K_GRID = tuple(
    sorted(
        {int(k) for k in cfg.k_grid if int(k) >= 2}
        | {int(PHASE3_K_FIXED)}
        | {int(v) for v in PHASE3_K_BY_MODEL.values()}
    )
)
PHASE3_K_DIAG_SAMPLE_N = 4000

geom_eval = GeometryClusteringEvaluator(
    random_state=cfg.random_state,
    silhouette_sample_size=PHASE3_SILHOUETTE_SAMPLE_SIZE,
)

if PHASE3_RESET or "phase3_by_model" not in globals() or not_
↳ isinstance(phase3_by_model, dict):
    phase3_by_model = {}

text_for_topic = reviews["clean_text"].fillna("").str.lower()
topic_proxy = np.select(
    [
        text_for_topic.str.
        ↳ contains(r"\b(entrega|prazo|atras|frete|transport)\w*", regex=True),
        text_for_topic.str.
        ↳ contains(r"\b(produit|qualidad|defeit|quebrad|avari)\w*", regex=True),
        text_for_topic.str.
        ↳ contains(r"\b(atendiment|suport|sac|respost|contat)\w*", regex=True),
        text_for_topic.str.contains(r"\b(troc|devolu|reembols|cancel)\w*",
        ↳ regex=True),
        text_for_topic.str.contains(r"\b(preco|valor|car[oa]|barat|cust)\w*",
        ↳ regex=True),
    ],
    ["delivery", "product", "support", "refund", "price"],
    default="other",
)
sentiment_labels = reviews["sentiment_label"].astype(str).to_numpy()

```

```

[45]: # --- Recompute clustering diagnostics/labels ---
if PHASE3_RECOMPUTE:
    for model_key in PHASE3_MODELS_THIS_RUN:
        if model_key not in model_run_order:

```

```

        raise ValueError(f"Unknown model_key in PHASE3_MODELS_THIS_RUN: {
↪{model_key}")
    print(f"Running Phase 3 for model: {model_key}")

    phase3_by_model[model_key] = run_phase3_for_model(
        model_key=model_key,
        embeddings_l2_by_model=embeddings_l2_by_model,
        sentiment_labels=sentiment_labels,
        topic_proxy=topic_proxy,
        cfg=cfg,
        geom_eval=geom_eval,
        k_grid=PHASE3_K_GRID,
        k_diag_sample_n=PHASE3_K_DIAG_SAMPLE_N,
        k_mode=PHASE3_K_MODE,
        k_fixed=PHASE3_K_FIXED,
        k_by_model=PHASE3_K_BY_MODEL,
        silhouette_sample_size=PHASE3_SILHOUETTE_SAMPLE_SIZE,
    )
# --- Option B: use cached artifacts (recommended for notebook reruns) ---
else:
    missing = [m for m in PHASE3_MODELS_THIS_RUN if m not in phase3_by_model]
    if missing:
        print(f"Missing cached Phase 3 artifacts for {missing}; computing only_
↪missing models.")
        for model_key in missing:
            phase3_by_model[model_key] = run_phase3_for_model(
                model_key=model_key,
                embeddings_l2_by_model=embeddings_l2_by_model,
                sentiment_labels=sentiment_labels,
                topic_proxy=topic_proxy,
                cfg=cfg,
                geom_eval=geom_eval,
                k_grid=PHASE3_K_GRID,
                k_diag_sample_n=PHASE3_K_DIAG_SAMPLE_N,
                k_mode=PHASE3_K_MODE,
                k_fixed=PHASE3_K_FIXED,
                k_by_model=PHASE3_K_BY_MODEL,
                silhouette_sample_size=PHASE3_SILHOUETTE_SAMPLE_SIZE,
            )
    else:
        print("Using cached Phase 3 artifacts loaded in cell 64.")

```

Using cached Phase 3 artifacts loaded in cell 64.

```

[46]: # --- Build summary tables and bind active model outputs ---
phase3_rows = []
for model_key in model_run_order:

```

```

    if model_key not in phase3_by_model:
        continue
    p3 = phase3_by_model[model_key]
    if not isinstance(p3.get("purity_table"), pd.DataFrame) or
    p3["purity_table"].empty:
        continue
    kmeans_row = p3["purity_table"].loc[p3["purity_table"]["algorithm"] ==
    "kmeans"].iloc[0]
    phase3_rows.append(
        {
            "model_key": model_key,
            "best_k": int(p3["best_k"]),
            "k_mode": str(p3.get("k_selection_mode", "cached")),
            "auto_k_ranked": int(p3.get("auto_k_ranked", p3["best_k"])),
            "kmeans_silhouette": float(p3["kmeans_final_metrics"].
    get("silhouette_cosine", np.nan)),
            "spectral_silhouette": float(p3["spectral_metrics"].
    get("silhouette_cosine", np.nan)),
            "kmeans_purity_topic_proxy":
    float(kmeans_row["purity_topic_proxy"]),
            "kmeans_purity_sentiment": float(kmeans_row["purity_sentiment"]),
        }
    )

phase3_model_comparison = pd.DataFrame(phase3_rows).sort_values("model_key").
    reset_index(drop=True)
kmeans_scores_combined = concat_by_field(phase3_by_model, model_run_order,
    "kmeans_scores")
purity_table_combined = concat_by_field(phase3_by_model, model_run_order,
    "purity_table")
stability_runs_combined = concat_by_field(phase3_by_model, model_run_order,
    "stability_runs")
stability_summary_combined = concat_by_field(phase3_by_model, model_run_order,
    "stability_summary")
cluster_size_combined = concat_by_field(phase3_by_model, model_run_order,
    "cluster_size_df")
k_selection_diagnostics = kmeans_scores_combined.copy()

if phase3_model_comparison.empty:
    k_selection_summary = pd.DataFrame(columns=["model_key", "best_k",
    "k_mode", "auto_k_ranked"])
else:
    k_selection_summary = (
        phase3_model_comparison[["model_key", "best_k", "k_mode",
    "auto_k_ranked"]]
        .sort_values("model_key")

```

```

        .reset_index(drop=True)
    )

    if cfg.model_key not in phase3_by_model:
        raise KeyError(
            f"Active model '{cfg.model_key}' has not been processed yet. "
            "Include it in PHASE3_MODELS_THIS_RUN and rerun this cell."
        )

    active3 = phase3_by_model[cfg.model_key]
    embeddings = active3["embeddings"]
    X = embeddings
    best_k = active3["best_k"]
    labels = active3["labels"]
    spectral_labels = active3["spectral_labels"]
    kmeans_scores = active3["kmeans_scores"]
    purity_table = active3["purity_table"]
    stability_runs = active3["stability_runs"]
    stability_summary = active3["stability_summary"]
    cluster_size_df = active3["cluster_size_df"]
    kmeans_final_metrics = active3["kmeans_final_metrics"]
    spectral_metrics = active3["spectral_metrics"]

```

```

[47]: done_models = [m for m in model_run_order if m in phase3_by_model]
    pending_models = [m for m in model_run_order if m not in phase3_by_model]

    print(f"Phase 3 recompute mode: {PHASE3_RECOMPUTE}")
    print(f"Phase 3 silhouette sample size: {PHASE3_SILHOUETTE_SAMPLE_SIZE}")
    print(f"K diagnostics grid: {PHASE3_K_GRID} | diagnostic sample: {
        PHASE3_K_DIAG_SAMPLE_N}")
    print(f"K selection mode: {PHASE3_K_MODE}")
    if PHASE3_K_MODE == "fixed":
        print(f"Fixed k: {PHASE3_K_FIXED}")
    elif PHASE3_K_MODE == "per_model":
        print(f"Per-model k map: {PHASE3_K_BY_MODEL}")
    print(f"Phase 3 stored models: {done_models}")
    if pending_models:
        print(f"Pending models: {pending_models}")
    print(f"Active model: {cfg.model_key} | best_k={best_k}")
    display(k_selection_summary)

```

```

Phase 3 recompute mode: False
Phase 3 silhouette sample size: 12000
K diagnostics grid: (2, 4, 6, 7, 8, 10, 12) | diagnostic sample: 4000
K selection mode: fixed
Fixed k: 7
Phase 3 stored models: ['bertimbau', 'multilingual_minilm', 'openclip_text']
Active model: multilingual_minilm | best_k=7

```

	model_key	best_k	k_mode	auto_k_ranked
0	bertimbau	7	cached	7
1	multilingual_minilm	7	cached	7
2	openclip_text	7	cached	7

```
[48]: print("Model-specific k selected automatically (ranked by silhouette +
      ↪Calinski-Harabasz + Davies-Bouldin).")
      if "k_selection_summary" in globals():
          display(k_selection_summary)
      display(phase3_model_comparison)
```

Model-specific k selected automatically (ranked by silhouette + Calinski-Harabasz + Davies-Bouldin).

	model_key	best_k	k_mode	auto_k_ranked
0	bertimbau	7	cached	7
1	multilingual_minilm	7	cached	7
2	openclip_text	7	cached	7

	model_key	best_k	k_mode	auto_k_ranked	kmeans_silhouette \
0	bertimbau	7	cached	7	0.385390
1	multilingual_minilm	7	cached	7	0.099294
2	openclip_text	7	cached	7	0.216980

	spectral_silhouette	kmeans_purity_topic_proxy	kmeans_purity_sentiment
0	NaN	0.865062	0.761963
1	-0.202394	0.562584	0.791725
2	NaN	0.873271	0.711004

1.3.2 3.2 Model comparison: sentiment vs topic purity

We compare topic and sentiment purity to understand what structure each embedding model captures.

```
[49]: display(phase3_model_comparison)
      display(kmeans_scores_combined.head(15))
      display(stability_summary_combined)
      display(purity_table_combined)

      fig_model_sil = px.bar(
          phase3_model_comparison,

          x="model_key",
          y="kmeans_silhouette",
          title="K-Means silhouette by model",
          height=360,
      )

      fig_model_purity = px.bar(
          phase3_model_comparison,
```

```

x="model_key",
y=["kmeans_purity_topic_proxy", "kmeans_purity_sentiment"],
barmode="group",
title="K-Means purity (topic proxy vs sentiment)",
height=380,
)
fig_model_sil.show()
fig_model_purity.show()

```

	model_key	best_k	k_mode	auto_k_ranked	kmeans_silhouette \
0	bertimbau	7	cached	7	0.385390
1	multilingual_minilm	7	cached	7	0.099294
2	openclip_text	7	cached	7	0.216980

	spectral_silhouette	kmeans_purity_topic_proxy	kmeans_purity_sentiment
0	NaN	0.865062	0.761963
1	-0.202394	0.562584	0.791725
2	NaN	0.873271	0.711004

	k	silhouette_cosine	calinski_harabasz	davies_bouldin	inertia \
0	2	0.197979	474.470483	2.868346	2387.087158
1	4	0.150629	299.021490	3.136215	2180.808594
2	6	0.108617	233.842916	3.070284	2065.669189
3	7	0.104494	210.140296	3.046212	2029.528931
4	8	0.106578	194.291734	2.997958	1991.792114
5	10	0.094553	169.737457	2.891396	1931.046997
6	12	0.097636	151.451488	2.850990	1883.540405

	rank_sil	rank_ch	rank_db	rank_sum	model_key	selected_k \
0	1.0	1.0	2.0	4.0	multilingual_minilm	7
1	2.0	2.0	7.0	11.0	multilingual_minilm	7
2	3.0	3.0	6.0	12.0	multilingual_minilm	7
3	5.0	4.0	5.0	14.0	multilingual_minilm	7
4	4.0	5.0	4.0	13.0	multilingual_minilm	7
5	7.0	6.0	3.0	16.0	multilingual_minilm	7
6	6.0	7.0	1.0	14.0	multilingual_minilm	7

	auto_k_ranked	selected_mode	n_rows_diag
0	2	fixed	4000
1	2	fixed	4000
2	2	fixed	4000
3	2	fixed	4000
4	2	fixed	4000
5	2	fixed	4000
6	2	fixed	4000

	model_key	k	seeds_tested	silhouette_mean	silhouette_std \
0	bertimbau	8	4	0.384988	0.000247
1	multilingual_minilm	7	4	0.099606	0.000705

2	openclip_text	8	4	0.216914	0.000376
---	---------------	---	---	----------	----------

	calinski_mean	calinski_std	davies_mean	davies_std	inertia_mean \
0	1332.259164	1.235332	3.021271	0.015761	3245.586609
1	1905.976833	6.778158	3.009474	0.036765	18648.872559
2	1516.625934	0.028405	2.411292	0.001895	674.704712

	inertia_std
0	4.244709
1	15.810885
2	0.000486

	model_key	algorithm	purity_sentiment	purity_topic_proxy
0	bertimbau	kmeans	0.761963	0.865062
1	bertimbau	spectral	NaN	NaN
2	multilingual_minilm	kmeans	0.791725	0.562584
3	multilingual_minilm	spectral	0.621927	0.378401
4	openclip_text	kmeans	0.711004	0.873271
5	openclip_text	spectral	NaN	NaN

Detailed interpretation moved to report; notebook keeps only execution outputs.

```
[50]: #compare cluster size distribution across models
fig_cluster_size = px.bar(
    cluster_size_combined,
    x="cluster_id",
    y="ratio",
    color="model_key",
    title="Cluster size distribution by model (k-means with common k)",
    height=400,
)
fig_cluster_size.update_layout(xaxis_title="Cluster ID", yaxis_title="Ratio of_
↳total samples")
fig_cluster_size.show()

#table of cluster size distribution across models
cluster_size_pivot = cluster_size_combined.pivot(index="cluster_id",
↳columns="model_key", values="ratio").fillna(0)
display(cluster_size_pivot)
```

model_key	bertimbau	multilingual_minilm	openclip_text
cluster_id			
0	0.065569	0.115049	0.211469
1	0.382282	0.132059	0.205287
2	0.079740	0.208904	0.182721
3	0.105532	0.174802	0.071818
4	0.140394	0.099680	0.220488
5	0.113943	0.136708	0.033646
6	0.046263	0.132797	0.053273

7

0.066278

0.000000

0.021299

1.3.3 3.3 Projection Extraction (PCA, UMAP, t-SNE)

We create comparable 2D and 3D projections for geometric inspection and cross-method diagnostics.

```
[51]: from sklearn.metrics import silhouette_score

# --- Projection setup (2D views): rationale and shared settings ---
projection_module = importlib.reload(projection_module)
projector = projection_module.ProjectionEngine(random_state=cfg.random_state)

HAS_UMAP = importlib.util.find_spec("umap") is not None
if not HAS_UMAP:
    print("UMAP is not installed; UMAP compute fallback will be skipped.")

PHASE3_DIR = DATA_DIR / "phase3_multi"
LOAD_COORDS_CACHE = True
SAVE_COORDS_CACHE = True
BUILD_FULL_METHODS = ("PCA",) # fast default: do not run full t-SNE/UMAP here

SENTIMENT_PALETTE = {
    "Negative": "#d73027",
    "Neutral": "#7f7f7f",
    "Positive": "#1a9850",
}
SENTIMENT_ORDER = ["Negative", "Neutral", "Positive"]
SENTIMENT_MARKERS = {"Negative": "X", "Neutral": "o", "Positive": "P"}
TOPIC_ORDER = ["delivery", "product", "support", "refund", "price"]
TOPIC_COLORS = {
    "delivery": "#1f77b4",
    "product": "#2ca02c",
    "support": "#ff7f0e",
    "refund": "#d62728",
    "price": "#9467bd",
}

projection_jobs_all = {
    "PCA": dict(method="pca", n_components=2),
    "tSNE": dict(method="tsne", n_components=2, perplexity=35),
}
if HAS_UMAP:
    projection_jobs_all["UMAP"] = dict(method="umap", n_components=2,
    ↪n_neighbors=20, min_dist=0.05)

tsne_iter_arg = "max_iter" if "max_iter" in inspect.signature(projector.tsne).
    ↪parameters else "n_iter"
projection_jobs_all["tSNE"][tsne_iter_arg] = 1200
```

```

build_methods = [m for m in BUILD_FULL_METHODS if m in projection_jobs_all]
if not build_methods:
    raise RuntimeError("No valid methods selected in BUILD_FULL_METHODS.")

y_sentiment_raw_full = reviews["sentiment_label"].astype(str).str.strip().str.
    ↪lower().to_numpy()
y_sentiment_full = (
    pd.Series(y_sentiment_raw_full)
    .map({"negative": "Negative", "neutral": "Neutral", "positive": "Positive"})
    .fillna("Neutral")
    .to_numpy()
)
y_topic_full = pd.Series(topic_proxy).astype(str).str.strip().str.lower().
    ↪to_numpy()
review_id_full = (
    reviews["review_id"].to_numpy()
    if "review_id" in reviews.columns
    else np.arange(len(reviews), dtype=int)
)
review_text_full = reviews["review_text"].astype(str).to_numpy()

```

```

[52]: # --- Projection coordinates: cache-first compute by model/method ---
projection_by_model = {}
projection_error_rows = []

for model_key in model_run_order:
    X_model = phase3_by_model[model_key]["embeddings"]
    n_rows = len(X_model)
    projection_by_model[model_key] = {}

    print(f"[{model_key}] Building methods: {build_methods} | rows={n_rows:,}")

    for proj_name in build_methods:
        kwargs = projection_jobs_all[proj_name]
        cache_path = PHASE3_DIR / model_key / f"coords_{proj_name.lower()}.npy"
        coords = None

        if LOAD_COORDS_CACHE and cache_path.exists():
            try:
                cached = np.load(cache_path)
                if cached.ndim == 2 and cached.shape[0] == n_rows:
                    coords = cached
                    print(f" - {proj_name}: loaded cache ({cache_path.name})")
            except Exception as exc:
                projection_error_rows.append(
                    {

```

```

        "model_key": model_key,
        "projection": proj_name,
        "error": f"cache load failed: {exc}",
    }
)

if coords is None:
    try:
        coords = projector.project(X_model, **kwargs)
        print(f" - {proj_name}: computed")

        if SAVE_COORDS_CACHE:
            cache_path.parent.mkdir(parents=True, exist_ok=True)
            np.save(cache_path, coords.astype(np.float32))
    except Exception as exc:
        projection_error_rows.append(
            {
                "model_key": model_key,
                "projection": proj_name,
                "error": str(exc),
            }
        )
        continue

    projection_by_model[model_key][proj_name] = coords

projection_error_df = pd.DataFrame(projection_error_rows)
if not projection_error_df.empty:
    print("Projection warnings:")
    display(projection_error_df)

```

```

[bertimbau] Building methods: ['PCA'] | rows=36,567
- PCA: loaded cache (coords_pca.npy)
[multilingual_minilm] Building methods: ['PCA'] | rows=36,567
- PCA: loaded cache (coords_pca.npy)
[openclip_text] Building methods: ['PCA'] | rows=36,567
- PCA: loaded cache (coords_pca.npy)

```

```

[53]: # --- Build plotting frames + projection-quality diagnostics ---
viz_by_model = {}
quality_rows = []

for model_key in model_run_order:
    X_model = phase3_by_model[model_key]["embeddings"]
    labels_model = ensure_model_labels(
        model_key=model_key,
        X_model=X_model,

```

```

        phase3_by_model=phase3_by_model,
        random_state=cfg.random_state,
        k_grid=tuple(cfg.k_grid),
    )
    viz_by_model[model_key] = {}

    for proj_name, coords in projection_by_model.get(model_key, {}).items():
        n_use = int(
            min(
                len(coords),
                len(labels_model),
                len(y_sentiment_raw_full),
                len(y_sentiment_full),
                len(y_topic_full),
                len(review_id_full),
                len(review_text_full),
            )
        )
        if n_use < len(coords):
            print(f"[{model_key} | {proj_name}] trimming rows for alignment:␣
↪coords={len(coords):,} -> {n_use:,}.")

            if n_use < 3:
                print(f"[{model_key} | {proj_name}] insufficient aligned rows␣
↪({n_use}) - skipped.")
                continue

        coords_use = coords[:n_use]
        labels_use = labels_model[:n_use]
        sent_raw_use = y_sentiment_raw_full[:n_use]
        sent_use = y_sentiment_full[:n_use]
        topic_use = y_topic_full[:n_use]
        rid_use = review_id_full[:n_use]
        rtext_use = review_text_full[:n_use]

        sil_sent = np.nan
        sil_topic = np.nan
        if len(np.unique(sent_raw_use)) > 1:
            sil_sent = float(silhouette_score(coords_use, sent_raw_use))
        if len(np.unique(topic_use)) > 1:
            sil_topic = float(silhouette_score(coords_use, topic_use))

        quality_rows.append(
            {
                "model_key": model_key,
                "projection": proj_name,
                "silhouette_sentiment": sil_sent,

```

```

        "silhouette_topic_proxy": sil_topic,
        "delta_topic_minus_sentiment": sil_topic - sil_sent,
        "n_rows": int(n_use),
    }
)

viz_by_model[model_key][proj_name] = pd.DataFrame(
    {
        "x": coords_use[:, 0],
        "y": coords_use[:, 1],
        "cluster": labels_use.astype(str),
        "sentiment": sent_use,
        "topic_proxy": topic_use,
        "review_id": rid_use,
        "review_text": rtext_use,
    }
)

proj_quality_df = pd.DataFrame(quality_rows).sort_values(["model_key",
↳ "projection"]).reset_index(drop=True)
if not proj_quality_df.empty:
    display(proj_quality_df)

```

	model_key	projection	silhouette_sentiment \
0	bertimbau	PCA	0.073727
1	multilingual_minilm	PCA	0.127935
2	openclip_text	PCA	0.045734

	silhouette_topic_proxy	delta_topic_minus_sentiment	n_rows
0	-0.102146	-0.175873	36567
1	-0.153211	-0.281146	36567
2	-0.071214	-0.116948	36567

```

[54]: # --- Bind active model projection objects used by following cells ---
proj_name_map = {"pca": "PCA", "umap": "UMAP", "tsne": "tSNE"}
selected_name = proj_name_map.get(cfg.projection_method, "PCA")

active_proj = projection_by_model.get(cfg.model_key, {})
if not active_proj:
    raise RuntimeError("No projections available for active model. Check_
↳ BUILD_FULL_METHODS and cache.")

if selected_name not in active_proj:
    selected_name = "PCA" if "PCA" in active_proj else list(active_proj.
↳ keys())[0]

projection_used = selected_name.lower()

```

```

projection_2d = active_proj
viz_dfs = viz_by_model[cfg.model_key]
coords_pca = active_proj.get("PCA")
coords_non_linear = active_proj[selected_name]

viz_pca = viz_dfs.get("PCA", viz_dfs[selected_name]).copy()
viz_nl = viz_dfs[selected_name].copy()

viz_pca["review_text"] = viz_pca["review_text"].astype(str)
viz_nl["review_text"] = viz_nl["review_text"].astype(str)

print(f"Active model: {cfg.model_key} | projection used: {selected_name}")
if coords_pca is not None:
    print(f"Active PCA coords: {coords_pca.shape} | active selected coords: {coords_non_linear.shape}")
else:
    print(f"Active selected coords: {coords_non_linear.shape}")

```

Active model: multilingual_minilm | projection used: PCA
Active PCA coords: (36567, 2) | active selected coords: (36567, 2)

3.3.1 PCA Method PCA provides a stable linear baseline for comparing cluster geometry across models.

```

[55]: METHOD_NAME = "PCA"
viz_pca_compare = collect_method_frame(
    METHOD_NAME,
    viz_by_model=viz_by_model,
    model_run_order=model_run_order,
    max_points=2500,
    random_state=cfg.random_state,
)

```

```

[56]: if viz_pca_compare.empty:
    print("No PCA frames found. Run shared projection cell first.")
else:
    plot_method_panels(
        viz_pca_compare,
        METHOD_NAME,
        model_run_order=model_run_order,
        sentiment_palette=SENTIMENT_PALETTE,
        sentiment_order=SENTIMENT_ORDER,
        sentiment_markers=SENTIMENT_MARKERS,
        topic_order=TOPIC_ORDER,
        topic_colors=TOPIC_COLORS,
    )

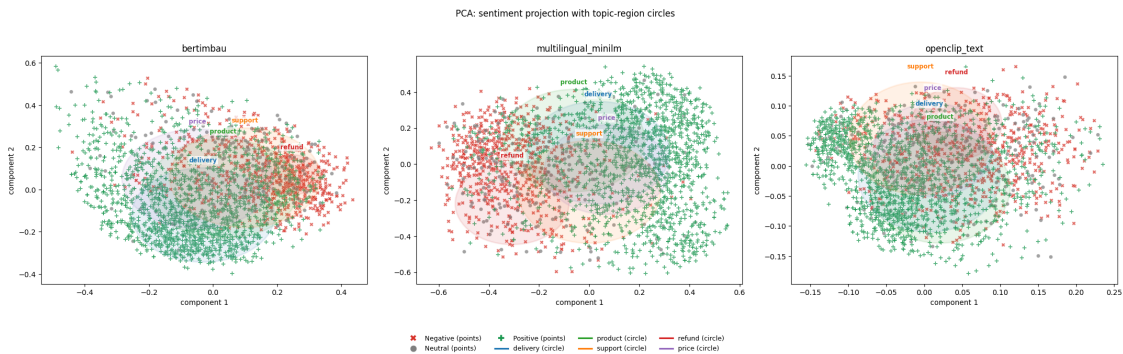
```

```

if "proj_quality_df" in globals() and isinstance(proj_quality_df, pd.DataFrame):
    display(proj_quality_df[proj_quality_df["projection"] == METHOD_NAME].
    ↪sort_values("model_key"))

if "projection_viz_exports" not in globals() or not
    ↪isinstance(projection_viz_exports, dict):
    projection_viz_exports = {}
if not viz_pca_compare.empty:
    projection_viz_exports[METHOD_NAME] = viz_pca_compare.copy()

```



	model_key	projection	silhouette_sentiment \
0	bertimbau	PCA	0.073727
1	multilingual_minilm	PCA	0.127935
2	openclip_text	PCA	0.045734

	silhouette_topic_proxy	delta_topic_minus_sentiment	n_rows
0	-0.102146	-0.175873	36567
1	-0.153211	-0.281146	36567
2	-0.071214	-0.116948	36567

```

[57]: # 3D PCA projection (cross-model)
METHOD_NAME_3D = "PCA"
PCA3D_MAX_ROWS = 2500
rng_pca3d = np.random.default_rng(cfg.random_state)

pca3d = pd.DataFrame()
reuse_existing_pca3d = False

if "projection3d_df" in globals() and isinstance(projection3d_df, pd.DataFrame)
    ↪and not projection3d_df.empty:
    pca3d_candidate = projection3d_df[projection3d_df["projection"] ==
    ↪METHOD_NAME_3D].copy()
    if not pca3d_candidate.empty:
        present_models = set(pca3d_candidate["model_key"].unique().tolist())

```

```

        if set(model_run_order).issubset(present_models):
            pca3d = pca3d_candidate
            reuse_existing_pca3d = True

if not reuse_existing_pca3d:
    pca3d_frames = []

    for model_key in model_run_order:
        X_model = phase3_by_model[model_key]["embeddings"]
        labels_model = phase3_by_model[model_key]["labels"]

        n_total = len(X_model)
        n_use = min(PCA3D_MAX_ROWS, n_total)
        if n_use < n_total:
            idx = np.sort(rng_pca3d.choice(n_total, size=n_use, replace=False))
        else:
            idx = np.arange(n_total)

        coords3d = projector.project(X_model[idx], method="pca", n_components=3)

        review_text_sub = reviews["review_text"].iloc[idx].astype(str).
        to_numpy()
        pca3d_frames.append(
            pd.DataFrame(
                {
                    "x": coords3d[:, 0],
                    "y": coords3d[:, 1],
                    "z": coords3d[:, 2],
                    "cluster": labels_model[idx].astype(str),
                    "sentiment": reviews["sentiment_label"].iloc[idx].
        to_numpy(),
                    "review_id": (
                        reviews["review_id"].iloc[idx].to_numpy()
                        if "review_id" in reviews.columns
                        else idx
                    ),
                    "review_text": review_text_sub,
                    "snippet": pd.Series(review_text_sub).str.slice(0, 140).
        to_numpy(),
                    "model_key": model_key,
                    "projection": METHOD_NAME_3D,
                }
            )
        )

    pca3d = pd.concat(pca3d_frames, ignore_index=True) if pca3d_frames else pd.
    to_numpy()
    DataFrame()

```

```

    if "projection3d_df" in globals() and isinstance(projection3d_df, pd.
↳DataFrame) and not projection3d_df.empty:
        projection3d_df = pd.concat(
            [projection3d_df[projection3d_df["projection"] != METHOD_NAME_3D],
↳pca3d],
            ignore_index=True,
        )
    else:
        projection3d_df = pca3d.copy()

```

```

[58]: if not pca3d.empty and "snippet" not in pca3d.columns:
    if "review_text" in pca3d.columns:
        pca3d["snippet"] = pca3d["review_text"].astype(str).str.slice(0, 140)
    else:
        pca3d["snippet"] = ""

if pca3d.empty:
    print("No PCA 3D frames available.")
else:
    for model_key in model_run_order:
        s = pca3d[pca3d["model_key"] == model_key]
        if s.empty:
            continue

        fig3d_overlay_b = px.scatter_3d(
            s,
            x="x",
            y="y",
            z="z",
            color="cluster",
            symbol="sentiment",
            hover_data={"review_id": True, "sentiment": True, "snippet": True},
            title=f"3D PCA overlay B (cluster color + sentiment marker) -
↳{model_key}",
            height=680,
        )
        fig3d_overlay_b.show()

```

Detailed interpretation moved to report; notebook keeps only execution outputs.

Detailed interpretation moved to report; notebook keeps only execution outputs.

3.3.2 t-SNE Method t-SNE emphasizes local neighborhood structure and helps inspect potential cluster boundaries.

```

[59]: # --- t-SNE 2D view (cache-first; sampled fallback) ---
METHOD_NAME = "tSNE"

```

```

VIZ_POINTS_PER_MODEL = 2500
TSNE_FALLBACK_MAX_ROWS = 3000
rng_tsne = np.random.default_rng(cfg.random_state)

# If missing, try cache first, then sampled fallback compute.
tsne_params = dict(method="tsne", n_components=2, perplexity=35)
tsne_iter_arg = "max_iter" if "max_iter" in inspect.signature(projector.tsne).
    ↪parameters else "n_iter"
tsne_params[tsne_iter_arg] = 1200

```

```

[60]: for model_key in model_run_order:
    if METHOD_NAME in viz_by_model.get(model_key, {}):
        continue

    X_model = phase3_by_model[model_key]["embeddings"]
    labels_model = phase3_by_model[model_key]["labels"]
    n_total = len(X_model)

    cache_path = DATA_DIR / "phase3_multi" / model_key / "coords_tsne.npy"
    coords = None
    idx = None

    if cache_path.exists():
        try:
            cached = np.load(cache_path)
            if cached.ndim == 2 and cached.shape[0] == n_total:
                coords = cached
                idx = np.arange(n_total)
                print(f"[{model_key}] t-SNE loaded from cache")
        except Exception:
            coords = None

    if coords is None:
        n_use = min(TSNE_FALLBACK_MAX_ROWS, n_total)
        if n_use < n_total:
            idx = np.sort(rng_tsne.choice(n_total, size=n_use, replace=False))
        else:
            idx = np.arange(n_total)

        print(f"[{model_key}] computing sampled t-SNE on {len(idx):,}/{n_total:
    ↪,} rows")
        coords = projector.project(X_model[idx], **tsne_params)

        sent_raw_sub = reviews["sentiment_label"].iloc[idx].astype(str).str.strip().
    ↪str.lower().to_numpy()
        sent_sub = (
            pd.Series(sent_raw_sub)

```

```

        .map({"negative": "Negative", "neutral": "Neutral", "positive": "Positive"})
        .fillna("Neutral")
        .to_numpy()
    )
    topic_sub = pd.Series(topic_proxy[idx]).astype(str).str.strip().str.lower().to_numpy()

    viz_by_model.setdefault(model_key, {})[METHOD_NAME] = pd.DataFrame(
        {
            "x": coords[:, 0],
            "y": coords[:, 1],
            "cluster": labels_model[idx].astype(str),
            "sentiment": sent_sub,
            "topic_proxy": topic_sub,
            "review_id": (
                reviews["review_id"].iloc[idx].to_numpy()
                if "review_id" in reviews.columns
                else idx
            ),
            "review_text": reviews["review_text"].iloc[idx].astype(str).to_numpy(),
        }
    )

    sil_sent = float(silhouette_score(coords, sent_raw_sub)) if len(np.unique(sent_raw_sub)) > 1 else np.nan
    sil_topic = float(silhouette_score(coords, topic_sub)) if len(np.unique(topic_sub)) > 1 else np.nan

    new_row = pd.DataFrame(
        [
            {
                "model_key": model_key,
                "projection": METHOD_NAME,
                "silhouette_sentiment": sil_sent,
                "silhouette_topic_proxy": sil_topic,
                "delta_topic_minus_sentiment": sil_topic - sil_sent,
                "n_rows": int(len(idx)),
            }
        ]
    )

    if "proj_quality_df" in globals() and isinstance(proj_quality_df, pd.DataFrame):
        proj_quality_df = pd.concat([proj_quality_df, new_row], ignore_index=True)

```

```

        proj_quality_df = (
            proj_quality_df
            .drop_duplicates(subset=["model_key", "projection"], keep="last")
            .sort_values(["model_key", "projection"])
            .reset_index(drop=True)
        )
    else:
        proj_quality_df = new_row

```

```

[bertimbau] t-SNE loaded from cache
[multilingual_minilm] t-SNE loaded from cache
[openclip_text] t-SNE loaded from cache

```

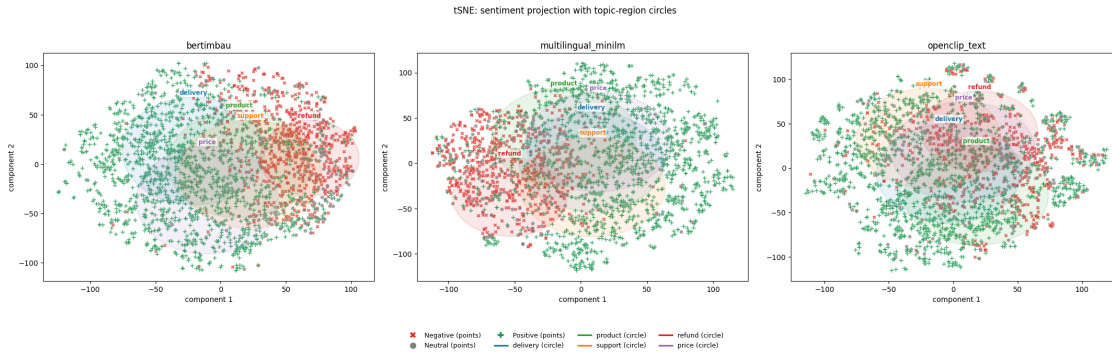
```

[61]: viz_tsne = collect_method_frame(
    METHOD_NAME,
    viz_by_model=viz_by_model,
    model_run_order=model_run_order,
    max_points=VIZ_POINTS_PER_MODEL,
    random_state=cfg.random_state,
)

if "projection_viz_exports" not in globals() or not_
↳ isinstance(projection_viz_exports, dict):
    projection_viz_exports = {}
if not viz_tsne.empty:
    projection_viz_exports[METHOD_NAME] = viz_tsne.copy()
if viz_tsne.empty:
    print("No t-SNE frames available.")
else:
    plot_method_panels(
        viz_tsne,
        METHOD_NAME,
        model_run_order=model_run_order,
        sentiment_palette=SENTIMENT_PALETTE,
        sentiment_order=SENTIMENT_ORDER,
        sentiment_markers=SENTIMENT_MARKERS,
        topic_order=TOPIC_ORDER,
        topic_colors=TOPIC_COLORS,
    )

if "proj_quality_df" in globals() and isinstance(proj_quality_df, pd.DataFrame):
    display(proj_quality_df[proj_quality_df["projection"] == METHOD_NAME].
↳ sort_values("model_key"))

```



	model_key	projection	silhouette_sentiment \
1	bertimbau	tSNE	0.120439
3	multilingual_minilm	tSNE	0.067316
5	openclip_text	tSNE	-0.013458

	silhouette_topic_proxy	delta_topic_minus_sentiment	n_rows
1	-0.174310	-0.294749	36567
3	-0.133787	-0.201104	36567
5	-0.118520	-0.105062	36567

```
[62]: if "projection3d_df" in globals() and isinstance(projection3d_df, pd.DataFrame):
    and not projection3d_df.empty:
        tsne3d = projection3d_df[projection3d_df["projection"] == METHOD_NAME]
        for model_key in model_run_order:
            s = tsne3d[tsne3d["model_key"] == model_key]
            if s.empty:
                continue

            fig3d_sent = px.scatter_3d(
                s,
                x="x", y="y", z="z",
                color="sentiment",
                hover_data={"review_id": True, "cluster": True},
                title=f"3D t-SNE sentiment ({model_key})",
                height=680,
            )
            fig3d_sent.show()

            fig3d_cluster = px.scatter_3d(
                s,
                x="x", y="y", z="z",
                color="cluster",
                hover_data={"review_id": True, "sentiment": True},
                title=f"3D t-SNE cluster ({model_key})",
```

```

        height=680,
    )
    fig3d_cluster.show()

```

Detailed interpretation moved to report; notebook keeps only execution outputs.

3.3.3 UMAP Method UMAP balances local and global structure and complements the PCA and t-SNE views.

```

[63]: # --- UMAP 2D view (cache-first; sampled fallback) ---
METHOD_NAME = "UMAP"
VIZ_POINTS_PER_MODEL = 2500
UMAP_FALLBACK_MAX_ROWS = 3000
rng_umap = np.random.default_rng(cfg.random_state)

has_umap_local = importlib.util.find_spec("umap") is not None

[64]: # Resolve UMAP coordinates per model (cache-first, sampled fallback).
umap_payloads = {}

for model_key in model_run_order:
    if METHOD_NAME in viz_by_model.get(model_key, {}):
        continue

    X_model = phase3_by_model[model_key]["embeddings"]
    labels_model = phase3_by_model[model_key]["labels"]
    n_total = len(X_model)

    cache_path = DATA_DIR / "phase3_multi" / model_key / "coords_umap.npy"
    coords = None
    idx = None

    if cache_path.exists():
        try:
            cached = np.load(cache_path)
            if cached.ndim == 2 and cached.shape[0] == n_total:
                coords = cached
                idx = np.arange(n_total)
                print(f"[{model_key}] UMAP loaded from cache")
        except Exception:
            coords = None

    if coords is None:
        if not has_umap_local:
            print(f"[{model_key}] UMAP unavailable (not installed) and no valid_
↪cache found.")
            continue

```

```

n_use = min(UMAP_FALLBACK_MAX_ROWS, n_total)
if n_use < n_total:
    idx = np.sort(rng_umap.choice(n_total, size=n_use, replace=False))
else:
    idx = np.arange(n_total)

print(f"[{model_key}] computing sampled UMAP on {len(idx):,}/{n_total:
↪,} rows")
coords = projector.project(
    X_model[idx],
    method="umap",
    n_components=2,
    n_neighbors=20,
    min_dist=0.05,
)

umap_payloads[model_key] = {"coords": coords, "idx": idx, "labels": ↵
↪labels_model}

```

[bertimbau] UMAP loaded from cache
[multilingual_minilm] UMAP loaded from cache
[openclip_text] UMAP loaded from cache

```

[65]: # Build UMAP plotting frames and projection-quality rows from prepared ↵
↪coordinates.
for model_key, payload in umap_payloads.items():
    coords = payload["coords"]
    idx = payload["idx"]
    labels_model = payload["labels"]

    sent_raw_sub = reviews["sentiment_label"].iloc[idx].astype(str).str.strip().
↪str.lower().to_numpy()
    sent_sub = (
        pd.Series(sent_raw_sub)
        .map({"negative": "Negative", "neutral": "Neutral", "positive": ↵
↪"Positive"})
        .fillna("Neutral")
        .to_numpy()
    )
    topic_sub = pd.Series(topic_proxy[idx]).astype(str).str.strip().str.lower().
↪to_numpy()

    viz_by_model.setdefault(model_key, {})[METHOD_NAME] = pd.DataFrame(
        {
            "x": coords[:, 0],
            "y": coords[:, 1],

```

```

        "cluster": labels_model[idx].astype(str),
        "sentiment": sent_sub,
        "topic_proxy": topic_sub,
        "review_id": (
            reviews["review_id"].iloc[idx].to_numpy()
            if "review_id" in reviews.columns
            else idx
        ),
        "review_text": reviews["review_text"].iloc[idx].astype(str).
        to_numpy(),
    }
)

sil_sent = float(silhouette_score(coords, sent_raw_sub)) if len(np.
unique(sent_raw_sub)) > 1 else np.nan
sil_topic = float(silhouette_score(coords, topic_sub)) if len(np.
unique(topic_sub)) > 1 else np.nan

new_row = pd.DataFrame(
    [
        {
            "model_key": model_key,
            "projection": METHOD_NAME,
            "silhouette_sentiment": sil_sent,
            "silhouette_topic_proxy": sil_topic,
            "delta_topic_minus_sentiment": sil_topic - sil_sent,
            "n_rows": int(len(idx)),
        }
    ]
)

if "proj_quality_df" in globals() and isinstance(proj_quality_df, pd.
Dataframe):
    proj_quality_df = pd.concat([proj_quality_df, new_row],
ignore_index=True)
    proj_quality_df = (
        proj_quality_df
        .drop_duplicates(subset=["model_key", "projection"], keep="last")
        .sort_values(["model_key", "projection"])
        .reset_index(drop=True)
    )
else:
    proj_quality_df = new_row

```

```

[66]: viz_umap = collect_method_frame(
    METHOD_NAME,
    viz_by_model=viz_by_model,
    model_run_order=model_run_order,

```

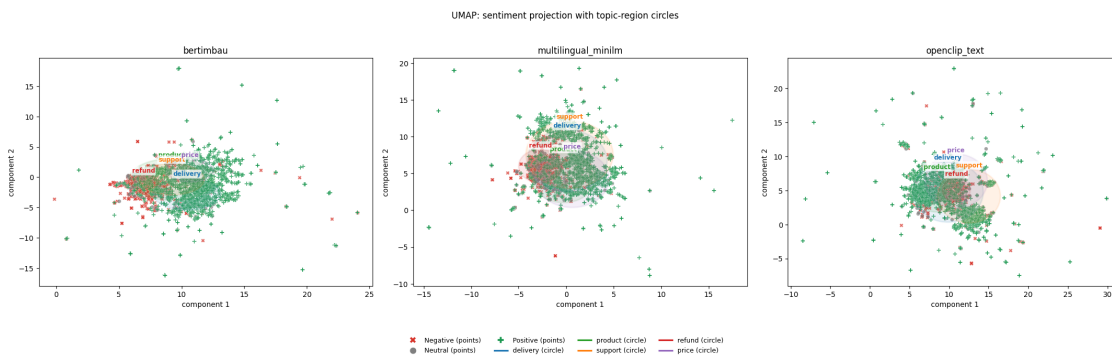
```

max_points=VIZ_POINTS_PER_MODEL,
random_state=cfg.random_state,
)

if "projection_viz_exports" not in globals() or not
↳ isinstance(projection_viz_exports, dict):
    projection_viz_exports = {}
if not viz_umap.empty:
    projection_viz_exports[METHOD_NAME] = viz_umap.copy()
if viz_umap.empty:
    print("No UMAP frames found. Install `umap-learn` or provide cached
↳ coords_umap.npy files.")
else:
    plot_method_panels(
        viz_umap,
        METHOD_NAME,
        model_run_order=model_run_order,
        sentiment_palette=SENTIMENT_PALETTE,
        sentiment_order=SENTIMENT_ORDER,
        sentiment_markers=SENTIMENT_MARKERS,
        topic_order=TOPIC_ORDER,
        topic_colors=TOPIC_COLORS,
    )

if "proj_quality_df" in globals() and isinstance(proj_quality_df, pd.DataFrame):
    display(proj_quality_df[proj_quality_df["projection"] == METHOD_NAME].
↳ sort_values("model_key"))

```



	model_key	projection	silhouette_sentiment \
1	bertimbau	UMAP	0.053428
4	multilingual_minilm	UMAP	-0.001603
7	openclip_text	UMAP	-0.069864

silhouette_topic_proxy	delta_topic_minus_sentiment	n_rows
------------------------	-----------------------------	--------

1	-0.246295	-0.299723	36567
4	-0.170172	-0.168570	36567
7	-0.161479	-0.091615	36567

```
[67]: # Optional 3D views (if 3D cell has been executed)
if "projection3d_df" in globals() and isinstance(projection3d_df, pd.DataFrame)
    and not projection3d_df.empty:
    umap3d = projection3d_df[projection3d_df["projection"] == METHOD_NAME]
    for model_key in model_run_order:
        s = umap3d[umap3d["model_key"] == model_key]
        if s.empty:
            continue

    fig3d_sent = px.scatter_3d(
        s,
        x="x", y="y", z="z",
        color="sentiment",
        hover_data={"review_id": True, "cluster": True},
        title=f"3D UMAP sentiment ({model_key})",
        height=680,
    )
    fig3d_sent.show()

    fig3d_cluster = px.scatter_3d(
        s,
        x="x", y="y", z="z",
        color="cluster",
        hover_data={"review_id": True, "sentiment": True},
        title=f"3D UMAP cluster ({model_key})",
        height=680,
    )
    fig3d_cluster.show()
```

Detailed interpretation moved to report; notebook keeps only execution outputs.

3.3.4 Model comparison This subsection summarizes projection-quality metrics across methods and embedding models.

```
[68]: # Projection-quality comparison across methods/models
fig_quality_topic = px.bar(
    proj_quality_df,
    x="projection",
    y="silhouette_topic_proxy",
    color="model_key",
    barmode="group",
    title="Projection quality: silhouette (topic proxy)",
    height=380,
)
```

```

fig_quality_topic.show()

fig_quality_delta = px.bar(
    proj_quality_df,
    x="projection",
    y="delta_topic_minus_sentiment",
    color="model_key",
    barmode="group",
    title="Projection quality: delta(topic - sentiment)",
    height=380,
)
fig_quality_delta.show()

```

Detailed interpretation moved to report; notebook keeps only execution outputs.

```

[69]: # Active model detailed sentiment map with text hover
fig_active_sent = px.scatter(
    viz_nl,
    x="x",
    y="y",
    color="cluster",
    color_discrete_map=SENTIMENT_PALETTE,
    category_orders={"sentiment": SENTIMENT_ORDER},
    hover_data={"review_id": True, "cluster": True, "review_text": True},
    title=f"Active model sentiment map with text ({cfg.model_key} +
    ↪{selected_name})",
    height=620,
)
fig_active_sent.show()

```

```

[70]: # --- Optional 3D projections for cross-model comparison ---
# 3D projections for cross-model comparison

if "projector" not in globals() or projector.__class__.__module__ != "src.
    ↪projection":
    projection_module = importlib.reload(projection_module)
    projector = projection_module.ProjectionEngine(random_state=cfg.
    ↪random_state)

PROJ3D_MAX_ROWS = 2500
RUN_HEAVY_3D = False # set True to compute tSNE/UMAP 3D maps
rng3d = np.random.default_rng(cfg.random_state)

HAS_UMAP_3D = importlib.util.find_spec("umap") is not None
if not HAS_UMAP_3D:
    print("UMAP is not installed; skipping 3D UMAP projection.")

```

```

projection_jobs_3d = {
    "PCA": dict(method="pca", n_components=3),
}
if RUN_HEAVY_3D:
    if HAS_UMAP_3D:
        projection_jobs_3d["UMAP"] = dict(method="umap", n_components=3,
↪n_neighbors=20, min_dist=0.05)

    tsne_params_3d = dict(method="tsne", n_components=3, perplexity=35)
    if "max_iter" in inspect.signature(projector.tsne).parameters:
        tsne_params_3d["max_iter"] = 1200
    else:
        tsne_params_3d["n_iter"] = 1200
    projection_jobs_3d["tSNE"] = tsne_params_3d

```

```

[71]: projection3d_by_model = {}
      projection3d_errors = []
      projection3d_frames = []

      for model_key in model_run_order:
          X_model = phase3_by_model[model_key]["embeddings"]
          labels_model = phase3_by_model[model_key]["labels"]

          n_total = len(X_model)
          n_use = min(PROJ3D_MAX_ROWS, n_total)

          if n_use < n_total:
              idx = np.sort(rng3d.choice(n_total, size=n_use, replace=False))
          else:
              idx = np.arange(n_total)

          X_sub = X_model[idx]

          sent_sub = reviews["sentiment_label"].iloc[idx].to_numpy()
          review_id_sub = (
              reviews["review_id"].iloc[idx].to_numpy()
              if "review_id" in reviews.columns
              else idx
          )
          cluster_sub = labels_model[idx].astype(str)

          projection3d_by_model[model_key] = {}

          print(f"[{model_key}] 3D projection on {len(X_sub):,}/{n_total:,} rows")

          for proj_name, kwargs in projection_jobs_3d.items():
              try:

```

```

coords3d = projector.project(X_sub, **kwargs)
projection3d_by_model[model_key][proj_name] = coords3d

df3d = pd.DataFrame(
    {
        "x": coords3d[:, 0],
        "y": coords3d[:, 1],
        "z": coords3d[:, 2],
        "cluster": cluster_sub,
        "sentiment": sent_sub,
        "review_id": review_id_sub,
        "model_key": model_key,
        "projection": proj_name,
    }
)
projection3d_frames.append(df3d)

except Exception as exc:
    projection3d_errors.append(
        {
            "model_key": model_key,
            "projection": proj_name,
            "error": str(exc),
        }
    )

projection3d_df = pd.concat(projection3d_frames, ignore_index=True) if
↳ projection3d_frames else pd.DataFrame()
projection3d_error_df = pd.DataFrame(projection3d_errors)

```

[bertimbau] 3D projection on 2,500/36,567 rows
[multilingual_minilm] 3D projection on 2,500/36,567 rows
[openclip_text] 3D projection on 2,500/36,567 rows

```

[72]: if not projection3d_error_df.empty:
        print("3D projection warnings:")
        display(projection3d_error_df)

    if projection3d_df.empty:
        raise RuntimeError("All 3D projection runs failed.")

```

```

[73]: # Sentiment-colored 3D maps (cross-model)
for proj_name in ["PCA", "UMAP", "tSNE"]:
    sub = projection3d_df[projection3d_df["projection"] == proj_name]
    if sub.empty:
        continue

```

```

for model_key in model_run_order:
    s = sub[sub["model_key"] == model_key]
    if s.empty:
        continue

    fig3d_sent = px.scatter_3d(
        s,
        x="x",
        y="y",
        z="z",
        color="sentiment",
        hover_data={"review_id": True, "cluster": True},
        title=f"3D sentiment map ({model_key} + {proj_name})",
        height=700,
    )
    fig3d_sent.show()

```

```

[74]: # Cluster-colored 3D maps (cross-model)
for proj_name in ["PCA", "UMAP", "tSNE"]:
    sub = projection3d_df[projection3d_df["projection"] == proj_name]
    if sub.empty:
        continue

    for model_key in model_run_order:
        s = sub[sub["model_key"] == model_key]
        if s.empty:
            continue

        fig3d_cluster = px.scatter_3d(
            s,
            x="x",
            y="y",
            z="z",
            color="cluster",
            hover_data={"review_id": True, "sentiment": True},
            title=f"3D cluster map ({model_key} + {proj_name})",
            height=700,
        )
        fig3d_cluster.show()

```

```

[75]: PHASE3_DIR = DATA_DIR / "phase3_multi"
PHASE3_DIR.mkdir(parents=True, exist_ok=True)

export_models = [m for m in model_run_order if m in phase3_by_model]
missing_models = [m for m in model_run_order if m not in phase3_by_model]
if missing_models:
    print(f"Skipping models not present in phase3_by_model: {missing_models}")

```

```

review_id_arr = reviews["review_id"].to_numpy() if "review_id" in reviews.
    ↳ columns else np.arange(len(reviews), dtype=int)
sentiment_arr = reviews["sentiment_label"].to_numpy()
topic_arr = np.asarray(topic_proxy)

```

```

[76]: for model_key in export_models:
    p3 = phase3_by_model[model_key]

    model_dir = PHASE3_DIR / model_key
    model_dir.mkdir(parents=True, exist_ok=True)

    labels_arr = np.asarray(p3["labels"]).astype(np.int32)
    spectral_arr = np.asarray(p3["spectral_labels"]).astype(np.int32)
    n_embed = int(len(p3["embeddings"])) if "embeddings" in p3 else
    ↳ int(len(labels_arr))

    n_use = int(min(n_embed, len(labels_arr), len(spectral_arr),
    ↳ len(review_id_arr), len(sentiment_arr), len(topic_arr)))
    if n_use < n_embed or n_use < len(labels_arr) or n_use < len(spectral_arr):
        print(
            f"[{model_key}] trimming export rows to {n_use:,} "
            f"(emb={n_embed:,}, labels={len(labels_arr):,},
    ↳ spectral={len(spectral_arr):,}, reviews={len(review_id_arr):,})"
        )

    labels_save = labels_arr[:n_use]
    spectral_save = spectral_arr[:n_use]

    np.save(model_dir / "labels_kmeans.npy", labels_save)
    np.save(model_dir / "labels_spectral.npy", spectral_save)

    p3["kmeans_scores"].to_csv(model_dir / "kmeans_scores.csv", index=False)
    p3["stability_runs"].to_csv(model_dir / "stability_runs.csv", index=False)
    p3["stability_summary"].to_csv(model_dir / "stability_summary.csv",
    ↳ index=False)
    p3["cluster_size_df"].to_csv(model_dir / "cluster_sizes.csv", index=False)
    p3["purity_table"].to_csv(model_dir / "purity_table.csv", index=False)

    pd.DataFrame([p3["kmeans_final_metrics"]]).to_csv(model_dir /
    ↳ "kmeans_final_metrics.csv", index=False)
    pd.DataFrame([p3["spectral_metrics"]]).to_csv(model_dir / "spectral_metrics.
    ↳ csv", index=False)

    label_index = pd.DataFrame(
        {

```

```

        "review_id": review_id_arr[:n_use],
        "kmeans_cluster": labels_save,
        "spectral_cluster": spectral_save,
        "sentiment_label": sentiment_arr[:n_use],
        "topic_proxy": topic_arr[:n_use],
    }
)
label_index.to_parquet(model_dir / "label_index.parquet", index=False)

if "projection_by_model" in globals() and model_key in projection_by_model:
    for proj_name, coords in projection_by_model[model_key].items():
        coords_arr = np.asarray(coords)
        if coords_arr.ndim == 2 and len(coords_arr) >= n_use:
            coords_arr = coords_arr[:n_use]
            np.save(model_dir / f"coords_{proj_name.lower()}.npy", coords_arr.
↳ astype(np.float32))

if "proj_quality_df" in globals() and not proj_quality_df.empty:
    proj_quality_df.query("model_key == @model_key").to_csv(
        model_dir / "projection_quality.csv", index=False
    )

manifest = {
    "model_key": model_key,
    "best_k": int(p3["best_k"]),
    "n_rows": int(n_use),
}
(model_dir / "phase3_manifest.json").write_text(
    json.dumps(manifest, ensure_ascii=False, indent=2),
    encoding="utf-8",
)

```

```

[77]: if "phase3_model_comparison" in globals() and not phase3_model_comparison.empty:
        phase3_model_comparison.to_csv(PHASE3_DIR / "phase3_model_comparison.csv",
↳ index=False)

if "proj_quality_df" in globals() and not proj_quality_df.empty:
    proj_quality_df.to_csv(PHASE3_DIR / "projection_quality_all_models.csv",
↳ index=False)

if "projection_error_df" in globals() and not projection_error_df.empty:
    projection_error_df.to_csv(PHASE3_DIR / "projection_errors.csv",
↳ index=False)

```

```

[78]: # Save section 3.4 method-level sampled views to avoid re-running visualization
↳ prep
VIZ_EXPORT_DIR = PHASE3_DIR / "projection_views"
VIZ_EXPORT_DIR.mkdir(parents=True, exist_ok=True)

```

```

projection_view_manifest = []
if "projection_viz_exports" in globals() and isinstance(projection_viz_exports, dict):
    for method_name, df_view in projection_viz_exports.items():
        if not isinstance(df_view, pd.DataFrame) or df_view.empty:
            continue

        method_path = VIZ_EXPORT_DIR / f"viz_{method_name.lower()}.parquet"
        cols_keep = [
            "x",
            "y",
            "cluster",
            "sentiment",
            "review_id",
            "model_key",
            "snippet",
            "topic_proxy",
        ]
        cols = [c for c in cols_keep if c in df_view.columns]
        df_view.loc[:, cols].to_parquet(method_path, index=False)

        projection_view_manifest.append(
            {
                "method": method_name,
                "n_rows": int(len(df_view)),
                "path": str(method_path),
            }
        )

if projection_view_manifest:
    pd.DataFrame(projection_view_manifest).to_csv(VIZ_EXPORT_DIR / "projection_view_manifest.csv", index=False)

```

```

[79]: print(f"Saved Phase 3 cache to: {PHASE3_DIR}")
      print(f"Saved section 3.4 projection views to: {VIZ_EXPORT_DIR}")

```

```

Saved Phase 3 cache to: C:\Users\HP\OneDrive - student.uw.edu.pl\PROJECTS\SEM
3-4\NLP-Semantic-P15\submission\data\phase3_multi
Saved section 3.4 projection views to: C:\Users\HP\OneDrive -
student.uw.edu.pl\PROJECTS\SEM 3-4\NLP-
Semantic-P15\submission\data\phase3_multi\projection_views

```

1.4 4 - Semantic Geometric Analysis

This section converts geometric outputs into direct evidence for RQ1 and RQ2 using dispersion and concentration diagnostics.

```
[80]: from itertools import combinations

# Phase 4 focuses on RQ evidence: dispersion, concentration, and uncertainty.
topic_modeler = ClusterTopicModeler(n_keywords=5, max_exemplars=5)
evaluator = GeometryEvaluator(random_state=cfg.random_state)

# --- RQ controls: island thresholds and bootstrap settings ---
NEG_ISLAND_Z_MIN = 2.0
NEG_ISLAND_DENSITY_Q = 0.40
NEG_HOMOPHILY_K = 20
N_BOOTSTRAP_RQ2 = 400
BOOT_CONF = 0.95

sent_arr = reviews["sentiment_label"].to_numpy()
neg_binary = (sent_arr == "negative").astype(np.int8)

phase4_by_model = {}
phase4_rows = []
negative_profile_all = []
bootstrap_ci_all = []
disp_sent_all = []
disp_topic_all = []
topic_table_all = []
```

```
[81]: # Build topic naming/table artifacts for one model.
def build_topic_outputs(model_key: str, Xm: np.ndarray, lm: np.ndarray):
    exemplars_m = topic_modeler.select_exemplars(
        texts=reviews["review_text"],
        embeddings=Xm,
        labels=lm,
    )
    keyword_names_m = topic_modeler.keyword_topic_names(
        texts=reviews["review_text"],
        labels=lm,
    )

    topic_name_map_m = keyword_names_m
    topic_labels_m = pd.Series(lm).map(topic_name_map_m).fillna("unlabeled").
    to_numpy()

    topic_table_m = topic_modeler.build_topic_table(
        texts=reviews["review_text"],
        embeddings=Xm,
        labels=lm,
        sentiment_labels=reviews["sentiment_label"],
    )
    topic_table_m["topic_name_keyword"] = topic_table_m["topic_name"]
```

```

    topic_table_m["topic_name"] = topic_table_m["cluster_id"].
    ↪map(topic_name_map_m).fillna(topic_table_m["topic_name_keyword"])
    topic_table_m["model_key"] = model_key

    return exemplars_m, topic_name_map_m, topic_labels_m, topic_table_m

```

```

[82]: # Compute dispersion and negative-island diagnostics for one model.
def build_negative_outputs(model_key: str, Xm: np.ndarray, lm: np.ndarray,
    ↪topic_labels_m: np.ndarray):
    disp_sent_m = dispersion_profile(Xm, sent_arr)
    disp_sent_m["model_key"] = model_key
    disp_sent_m["label_set"] = "sentiment"

    disp_topic_m = dispersion_profile(Xm, topic_labels_m)
    disp_topic_m["model_key"] = model_key
    disp_topic_m["label_set"] = "topic"

    neg_profile_m = evaluator.negative_profile(
        embeddings=Xm,
        labels=lm,
        sentiments=reviews["sentiment_label"],
    ).copy()
    neg_profile_m["model_key"] = model_key

    global_neg_m = float(neg_binary.mean())
    se = np.sqrt(np.clip((global_neg_m * (1.0 - global_neg_m)) /
    ↪neg_profile_m["size"].to_numpy(), 1e-12, None))
    neg_profile_m["z_negative"] = (neg_profile_m["negative_ratio"].to_numpy() -
    ↪global_neg_m) / se

    density_cut_m = float(neg_profile_m["dispersion"].
    ↪quantile(NEG_ISLAND_DENSITY_Q))
    neg_profile_m["dense_flag"] = neg_profile_m["dispersion"] <= density_cut_m
    neg_profile_m["island_flag"] = (neg_profile_m["z_negative"] >=
    ↪NEG_ISLAND_Z_MIN) & neg_profile_m["dense_flag"]

    island_count_m = int(neg_profile_m["island_flag"].sum())
    island_mass_m = float(neg_profile_m.loc[neg_profile_m["island_flag"],
    ↪"size"].sum() / len(reviews))

    hom_share_m, _, hom_excess_m = negative_homophily(Xm, sent_arr,
    ↪k=NEG_HOMOPHILY_K)

    boot_m = evaluator.bootstrap_profile(
        labels=lm,
        sentiments=reviews["sentiment_label"],

```

```

        n_bootstrap=400,
        confidence=0.95,
    )
    boot_m["model_key"] = model_key

    return disp_sent_m, disp_topic_m, neg_profile_m, boot_m, global_neg_m,
    hom_share_m, hom_excess_m, island_count_m, island_mass_m

```

```

[83]: # Wrapper to assemble all RQ metrics for one model.
def compute_phase4_for_model(model_key: str) -> dict:
    Xm = phase3_by_model[model_key]["embeddings"]
    lm = phase3_by_model[model_key]["labels"].astype(int)

    exemplars_m, topic_name_map_m, topic_labels_m, topic_table_m =
    build_topic_outputs(model_key, Xm, lm)
    (
        disp_sent_m,
        disp_topic_m,
        neg_profile_m,
        boot_m,
        global_neg_m,
        hom_share_m,
        hom_excess_m,
        island_count_m,
        island_mass_m,
    ) = build_negative_outputs(model_key, Xm, lm, topic_labels_m)

    kmeans_row = phase3_by_model[model_key]["purity_table"].query("algorithm ==
    'kmeans'").iloc[0]
    cluster_neg_std_m = float(topic_table_m["negative_ratio"].std())
    rq_signal_m = "islands" if (cluster_neg_std_m >= 0.12 and island_count_m >
    0) else "diffuse"

    row = {
        "model_key": model_key,
        "n_clusters": int(topic_table_m["cluster_id"].nunique()),
        "global_negative_ratio": global_neg_m,
        "cluster_negative_ratio_std": cluster_neg_std_m,
        "negative_homophily_k20": hom_share_m,
        "negative_homophily_excess_k20": hom_excess_m,
        "negative_island_count": island_count_m,
        "negative_island_mass": island_mass_m,
        "topic_dispersion_mean":
    float(disp_topic_m["mean_pairwise_cosine_distance"].mean()),
        "sentiment_dispersion_mean":
    float(disp_sent_m["mean_pairwise_cosine_distance"].mean()),
        "kmeans_purity_topic_proxy": float(kmeans_row["purity_topic_proxy"]),
    }

```

```

        "kmeans_purity_sentiment": float(kmeans_row["purity_sentiment"]),
        "rq_signal": rq_signal_m,
    }

    return {
        "exemplars": exemplars_m,
        "topic_name_map": topic_name_map_m,
        "topic_labels": topic_labels_m,
        "topic_table": topic_table_m,
        "disp_sent": disp_sent_m,
        "disp_topic": disp_topic_m,
        "negative_profile": neg_profile_m,
        "bootstrap_ci": boot_m,
        "global_neg_ratio": global_neg_m,
        "cluster_neg_std": cluster_neg_std_m,
        "homophily_share": hom_share_m,
        "homophily_excess": hom_excess_m,
        "island_count": island_count_m,
        "island_mass": island_mass_m,
        "rq_signal": rq_signal_m,
        "phase4_row": row,
    }

```

```

[84]: # --- Compute RQ metrics per model and aggregate outputs ---
for model_key in model_run_order:
    model_out = compute_phase4_for_model(model_key)
    phase4_by_model[model_key] = model_out

    phase4_rows.append(model_out["phase4_row"])
    negative_profile_all.append(model_out["negative_profile"])
    bootstrap_ci_all.append(model_out["bootstrap_ci"])
    disp_sent_all.append(model_out["disp_sent"])
    disp_topic_all.append(model_out["disp_topic"])
    topic_table_all.append(model_out["topic_table"])

phase4_model_comparison = pd.DataFrame(phase4_rows).sort_values("model_key").
    ↪reset_index(drop=True)
rq_model_comparison = phase4_model_comparison[
    [
        "model_key",
        "rq_signal",
        "cluster_negative_ratio_std",
        "negative_homophily_k20",
        "negative_homophily_excess_k20",
        "negative_island_count",
        "negative_island_mass",
        "kmeans_purity_topic_proxy",
    ]

```

```

        "kmeans_purity_sentiment",
        "topic_dispersion_mean",
        "sentiment_dispersion_mean",
    ]
].copy()

negative_profile_combined = pd.concat(negative_profile_all, ignore_index=True)
bootstrap_ci_combined = pd.concat(bootstrap_ci_all, ignore_index=True)
disp_sent_combined = pd.concat(disp_sent_all, ignore_index=True)
disp_topic_combined = pd.concat(disp_topic_all, ignore_index=True)
topic_table_combined = pd.concat(topic_table_all, ignore_index=True)

```

```

[85]: # --- RQ2: pairwise bootstrap deltas for cluster_negative_ratio_std ---
rng = np.random.default_rng(cfg.random_state)
n_rows = len(reviews)
boot_indices = [rng.integers(0, n_rows, size=n_rows) for _ in
    ↪range(N_BOOTSTRAP_RQ2)]

cluster_neg_std_boot = {}
cluster_neg_std_boot_rows = []
alpha = 1.0 - BOOT_CONF

for model_key in model_run_order:
    lm = phase3_by_model[model_key]["labels"].astype(int)
    boot_vals = bootstrap_cluster_neg_std(lm, neg_binary, boot_indices)
    cluster_neg_std_boot[model_key] = boot_vals
    cluster_neg_std_boot_rows.append(
        {
            "model_key": model_key,
            "metric": "cluster_negative_ratio_std",
            "estimate": float(boot_vals.mean()),
            "ci_low": float(np.quantile(boot_vals, alpha / 2.0)),
            "ci_high": float(np.quantile(boot_vals, 1.0 - alpha / 2.0)),
            "n_bootstrap": int(N_BOOTSTRAP_RQ2),
            "confidence": float(BOOT_CONF),
        }
    )

cluster_neg_std_bootstrap_ci = pd.DataFrame(cluster_neg_std_boot_rows).
    ↪sort_values("model_key").reset_index(drop=True)

pairwise_rows = []
for model_a, model_b in combinations(model_run_order, 2):
    delta = cluster_neg_std_boot[model_a] - cluster_neg_std_boot[model_b]
    ci_low = float(np.quantile(delta, alpha / 2.0))
    ci_high = float(np.quantile(delta, 1.0 - alpha / 2.0))

```

```

    if ci_low > 0:
        stronger = model_a
    elif ci_high < 0:
        stronger = model_b
    else:
        stronger = "inconclusive"

    pairwise_rows.append(
        {
            "model_a": model_a,
            "model_b": model_b,
            "metric": "cluster_negative_ratio_std",
            "delta_mean": float(delta.mean()),
            "ci_low": ci_low,
            "ci_high": ci_high,
            "stronger_islandization": stronger,
            "n_bootstrap": int(N_BOOTSTRAP_RQ2),
            "confidence": float(BOOT_CONF),
        }
    )

rq2_pairwise_delta_ci = pd.DataFrame(pairwise_rows).sort_values(["model_a",
↳ "model_b"]).reset_index(drop=True)

```

```

[86]: # Bind active-model tables used by the display cells below.
active4 = phase4_by_model[cfg.model_key]
exemplars = active4["exemplars"]
topic_name_map = active4["topic_name_map"]

reviews = reviews.reset_index(drop=True).copy()
reviews["cluster_kmeans"] = phase3_by_model[cfg.model_key]["labels"].astype(int)
reviews["topic_label"] = active4["topic_labels"]

topic_table = active4["topic_table"].copy()
disp_sent = active4["disp_sent"]
disp_topic = active4["disp_topic"]
negative_profile = active4["negative_profile"]
bootstrap_ci = active4["bootstrap_ci"]
global_neg_ratio = active4["global_neg_ratio"]
cluster_neg_std = active4["cluster_neg_std"]
rq_signal = active4["rq_signal"]
phase4_summary = pd.DataFrame([active4["phase4_row"]])

[87]: print(f"Phase 4 computed for models: {model_run_order}")
print(f"Active model: {cfg.model_key} | rq_signal={rq_signal}")
display(phase4_model_comparison)
display(cluster_neg_std_bootstrap_ci)

```

```
display(rq2_pairwise_delta_ci)
```

Phase 4 computed for models: ['bertimbau', 'multilingual_minilm', 'openclip_text']

Active model: multilingual_minilm | rq_signal=diffuse

	model_key	n_clusters	global_negative_ratio	\
0	bertimbau	7	0.28925	
1	multilingual_minilm	7	0.28925	
2	openclip_text	7	0.28925	

	cluster_negative_ratio_std	negative_homophily_k20	\
0	0.255537	0.725267	
1	0.286822	0.703101	
2	0.182565	0.595306	

	negative_homophily_excess_k20	negative_island_count	negative_island_mass	\
0	0.436017	1	0.234446	
1	0.413851	0	0.000000	
2	0.306056	1	0.143545	

	topic_dispersion_mean	sentiment_dispersion_mean	\
0	0.317300	0.376350	
1	0.498476	0.647222	
2	0.047085	0.054793	

	kmeans_purity_topic_proxy	kmeans_purity_sentiment	rq_signal
0	0.865062	0.761963	islands
1	0.562584	0.791725	diffuse
2	0.873271	0.711004	islands

	model_key	metric	estimate	ci_low	\
0	bertimbau	cluster_negative_ratio_std	0.255524	0.251538	
1	multilingual_minilm	cluster_negative_ratio_std	0.286728	0.283132	
2	openclip_text	cluster_negative_ratio_std	0.182629	0.177576	

	ci_high	n_bootstrap	confidence
0	0.259341	400	0.95
1	0.290314	400	0.95
2	0.187575	400	0.95

	model_a	model_b	metric	\
0	bertimbau	multilingual_minilm	cluster_negative_ratio_std	
1	bertimbau	openclip_text	cluster_negative_ratio_std	
2	multilingual_minilm	openclip_text	cluster_negative_ratio_std	

	delta_mean	ci_low	ci_high	stronger_islandization	n_bootstrap	\
0	-0.031204	-0.034329	-0.028050	multilingual_minilm	400	
1	0.072895	0.068785	0.078048	bertimbau	400	

2	0.104098	0.099918	0.108738	multilingual_minilm	400
---	----------	----------	----------	---------------------	-----

	confidence
0	0.95
1	0.95
2	0.95

1.4.1 4.1 Dispersion profile: topic vs sentiment

We compare within-group dispersion for topic-based and sentiment-based label partitions.

```
[88]: disp_compare_model = phase4_model_comparison[
    ["model_key", "topic_dispersion_mean", "sentiment_dispersion_mean"]
].copy()
disp_compare_model["delta_topic_minus_sentiment"] = (
    disp_compare_model["topic_dispersion_mean"] -
    disp_compare_model["sentiment_dispersion_mean"]
)

display(disp_compare_model.sort_values("model_key").reset_index(drop=True))
```

	model_key	topic_dispersion_mean	sentiment_dispersion_mean \
0	bertimbau	0.317300	0.376350
1	multilingual_minilm	0.498476	0.647222
2	openclip_text	0.047085	0.054793

	delta_topic_minus_sentiment
0	-0.059050
1	-0.148746
2	-0.007709

```
[89]: display(disp_sent)
display(disp_topic)

fig_disp = px.bar(
    disp_compare_model,
    x="model_key",
    y=["topic_dispersion_mean", "sentiment_dispersion_mean"],
    barmode="group",
    title="Mean dispersion by model: topic vs sentiment",
    height=380,
)
fig_disp.show()
```

	group	mean_pairwise_cosine_distance	n_samples	model_key \
0	neutral	0.700303	3248	multilingual_minilm
1	negative	0.643127	10577	multilingual_minilm
2	positive	0.598235	22742	multilingual_minilm

	label_set
0	sentiment
1	sentiment
2	sentiment

	group \
0	não de produto veio que
1	não recebi produto não recebi ainda
2	prazo antes do prazo entrega antes do
3	muito recomendo gostei tudo bom
4	loja compra recomendo muito comprar
5	produto prazo do prazo do antes
6	produto qualidade bom muito ótimo

	mean_pairwise_cosine_distance	n_samples	model_key	label_set
0	0.677341	6392	multilingual_minilm	topic
1	0.545313	7639	multilingual_minilm	topic
2	0.507105	4207	multilingual_minilm	topic
3	0.490178	4999	multilingual_minilm	topic
4	0.486832	3645	multilingual_minilm	topic
5	0.397568	4856	multilingual_minilm	topic
6	0.384995	4829	multilingual_minilm	topic

Detailed interpretation moved to report; notebook keeps only execution outputs.

1.4.2 4.2 Negative concentration, island flags, and bootstrap confidence intervals

We quantify cluster-level negativity concentration, island flags, and uncertainty with bootstrap intervals.

```
[90]: display(negative_profile)
display(bootstrap_ci)

island_clusters_active = negative_profile.loc[negative_profile["island_flag"]].
    ↪copy()
display(island_clusters_active)

# Add sentiment/topic composition so Section 4 is not only negative-ratio
    ↪centric.
cluster_sentiment = evaluator.sentiment_by_cluster(
    labels=phase3_by_model[cfg.model_key]["labels"],
    sentiments=reviews["sentiment_label"],
).fillna(0.0)

for c in ["negative", "neutral", "positive"]:
    if c not in cluster_sentiment.columns:
        cluster_sentiment[c] = 0.0

cluster_sentiment = cluster_sentiment.rename(
```

```

        columns={
            "negative": "sent_neg",
            "neutral": "sent_neu",
            "positive": "sent_pos",
        }
    )

    cluster_topic = (
        reviews.groupby(["cluster_kmeans", "topic_label"]).size().
        ↪rename("topic_count").reset_index()
    )
    cluster_topic["topic_share"] = cluster_topic["topic_count"] / cluster_topic.
    ↪groupby("cluster_kmeans")["topic_count"].transform("sum")

    top_topics = (
        cluster_topic.sort_values(["cluster_kmeans", "topic_count"],
        ↪ascending=[True, False])
        .groupby("cluster_kmeans")
        .head(3)
        .copy()
    )
    top_topics["topic_item"] = top_topics.apply(
        lambda r: f"{r['topic_label']} ({100.0 * r['topic_share']:.1f}%)",
        axis=1,
    )

    top_topic_summary = (
        top_topics.groupby("cluster_kmeans")["topic_item"]
        .apply(lambda x: "; ".join(x))
        .reset_index(name="top_topics")
    )

    cluster_profile = (
        negative_profile.merge(cluster_sentiment, on="cluster_id", how="left")
        .merge(top_topic_summary, left_on="cluster_id", right_on="cluster_kmeans",
        ↪how="left")
        .drop(columns=["cluster_kmeans"], errors="ignore")
        .sort_values("negative_ratio", ascending=False)
        .reset_index(drop=True)
    )

    display(cluster_profile)

    fig_neg = px.bar(
        negative_profile.sort_values("negative_ratio", ascending=False),
        x="cluster_id",
        y="negative_ratio",

```

```

        color="island_flag",
        title="Negative ratio by cluster (active model)",
        height=380,
    )
fig_neg.show()

sent_long = (
    cluster_sentiment[["cluster_id", "sent_neg", "sent_neu", "sent_pos"]]
    .melt(id_vars="cluster_id", var_name="sentiment", value_name="share")
)
fig_sent_mix = px.bar(
    sent_long,
    x="cluster_id",
    y="share",
    color="sentiment",
    barmode="stack",
    title="Sentiment mix by cluster (active model)",
    height=380,
)
fig_sent_mix.show()

```

	cluster_id	size	mean_cosine_similarity_to_centroid	dispersion \
0	3	6392	0.568124	0.431876
1	2	7639	0.674358	0.325642
2	0	4207	0.702151	0.297849
3	5	4999	0.714087	0.285913
4	4	3645	0.716451	0.283549
5	6	4856	0.776217	0.223783
6	1	4829	0.784273	0.215727

	negative_ratio	model_key	z_negative	dense_flag	island_flag
0	0.518304	multilingual_minilm	40.388869	False	False
1	0.752062	multilingual_minilm	89.212915	False	False
2	0.152127	multilingual_minilm	-19.615519	False	False
3	0.041208	multilingual_minilm	-38.678630	False	False
4	0.083676	multilingual_minilm	-27.372915	True	False
5	0.060956	multilingual_minilm	-35.086451	True	False
6	0.014910	multilingual_minilm	-42.045798	True	False

	cluster_id	estimate	ci_low	ci_high	n	model_key
0	2	0.752062	0.741982	0.762145	7639	multilingual_minilm
1	3	0.518304	0.505624	0.529580	6392	multilingual_minilm
2	0	0.152127	0.142619	0.164256	4207	multilingual_minilm
3	4	0.083676	0.073519	0.091907	3645	multilingual_minilm
4	6	0.060956	0.054160	0.067962	4856	multilingual_minilm
5	5	0.041208	0.036207	0.047009	4999	multilingual_minilm
6	1	0.014910	0.011390	0.018228	4829	multilingual_minilm

Empty DataFrame

Columns: [cluster_id, size, mean_cosine_similarity_to_centroid, dispersion,
 ↪negative_ratio, model_key, z_negative, dense_flag, island_flag]

Index: []

	cluster_id	size	mean_cosine_similarity_to_centroid	dispersion \
0	2	7639	0.674358	0.325642
1	3	6392	0.568124	0.431876
2	0	4207	0.702151	0.297849
3	4	3645	0.716451	0.283549
4	6	4856	0.776217	0.223783
5	5	4999	0.714087	0.285913
6	1	4829	0.784273	0.215727

	negative_ratio	model_key	z_negative	dense_flag	island_flag \
0	0.752062	multilingual_minilm	89.212915	False	False
1	0.518304	multilingual_minilm	40.388869	False	False
2	0.152127	multilingual_minilm	-19.615519	False	False
3	0.083676	multilingual_minilm	-27.372915	True	False
4	0.060956	multilingual_minilm	-35.086451	True	False
5	0.041208	multilingual_minilm	-38.678630	False	False
6	0.014910	multilingual_minilm	-42.045798	True	False

	sent_neg	sent_neu	sent_pos \
0	0.752062	0.108129	0.139809
1	0.518304	0.203066	0.278630
2	0.152127	0.083908	0.763965
3	0.083676	0.049657	0.866667
4	0.060956	0.050453	0.888591
5	0.041208	0.039408	0.919384
6	0.014910	0.030648	0.954442

	top_topics
0	não recebi produto não recebi ainda (100.0%)
1	não de produto veio que (100.0%)
2	prazo antes do prazo entrega antes do (100.0%)
3	loja compra recomendo muito comprar (100.0%)
4	produto prazo do prazo do antes (100.0%)
5	muito recomendo gostei tudo bom (100.0%)
6	produto qualidade bom muito ótimo (100.0%)

Detailed interpretation moved to report; notebook keeps only execution outputs.

1.4.3 4.3 Research question results

This subsection consolidates model-level evidence and states the final answers for RQ1 and RQ2.

```
[91]: active_row = phase4_model_comparison.loc[phase4_model_comparison["model_key"]  

  ↪== cfg.model_key].iloc[0]
```

```

print(f"Active model: {cfg.model_key}")
print(f"Global negative ratio: {active_row['global_negative_ratio']:.3f}")
print(f"Cluster negative std: {active_row['cluster_negative_ratio_std']:.3f}")
print(f"Negative homophily@{NEG_HOMOPHILY_K}:␣
↪{active_row['negative_homophily_k20']:.3f}")
print(f"Negative homophily excess@{NEG_HOMOPHILY_K}:␣
↪{active_row['negative_homophily_excess_k20']:.3f}")
print(f"Negative island count: {int(active_row['negative_island_count'])}")
print(f"Negative island mass: {active_row['negative_island_mass']:.3f}")
print(f"RQ1 answer (active model): {active_row['rq_signal']}")

print("\nRQ1 summary across models:")
display(rq_model_comparison)

print("\nRQ2 pairwise bootstrap deltas (cluster_negative_ratio_std):")
display(rq2_pairwise_delta_ci)

```

```

Active model: multilingual_minilm
Global negative ratio: 0.289
Cluster negative std: 0.287
Negative homophily@20: 0.703
Negative homophily excess@20: 0.414
Negative island count: 0
Negative island mass: 0.000
RQ1 answer (active model): diffuse

```

RQ1 summary across models:

	model_key	rq_signal	cluster_negative_ratio_std	\
0	bertimbau	islands	0.255537	
1	multilingual_minilm	diffuse	0.286822	
2	openclip_text	islands	0.182565	

	negative_homophily_k20	negative_homophily_excess_k20	\
0	0.725267	0.436017	
1	0.703101	0.413851	
2	0.595306	0.306056	

	negative_island_count	negative_island_mass	kmeans_purity_topic_proxy	\
0	1	0.234446	0.865062	
1	0	0.000000	0.562584	
2	1	0.143545	0.873271	

	kmeans_purity_sentiment	topic_dispersion_mean	sentiment_dispersion_mean
0	0.761963	0.317300	0.376350
1	0.791725	0.498476	0.647222
2	0.711004	0.047085	0.054793

RQ2 pairwise bootstrap deltas (cluster_negative_ratio_std):

	model_a	model_b	metric \
0	bertimbau	multilingual_minilm	cluster_negative_ratio_std
1	bertimbau	openclip_text	cluster_negative_ratio_std
2	multilingual_minilm	openclip_text	cluster_negative_ratio_std

	delta_mean	ci_low	ci_high	stronger_islandization	n_bootstrap \
0	-0.031204	-0.034329	-0.028050	multilingual_minilm	400
1	0.072895	0.068785	0.078048	bertimbau	400
2	0.104098	0.099918	0.108738	multilingual_minilm	400

	confidence
0	0.95
1	0.95
2	0.95